

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Сибирский государственный индустриальный университет»
Кафедра автоматизации и информационных систем

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

бакалаврская работа

**Разработка web-приложения для малого предприятия сферы
предоставления конно-спортивных услуг
(клиентская часть)**

Обучающийся: Орлов Данил Сергеевич

Группа: ИС-22

Руководитель ВКР: Кулаков С. М.

Руководитель практики: Тараборина Е. Н.

Профильная организация: ИП Орлова Н. И.

Новокузнецк

2026

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ

Тема выпускной квалификационной работы: «Разработка web-приложения для малого предприятия сферы предоставления конно-спортивных услуг (клиентская часть)».

Исходными данными для разработки являются сведения о деятельности ИП Орлова Н. И. в г. Гурьевске, материалы преддипломной практики, требования к клиентскому интерфейсу, макеты страниц, перечень услуг, правила посещения, требования безопасности и ограничения, связанные с использованием лошадей как ограниченного ресурса.

Цель работы - разработать клиентскую часть web-приложения, обеспечивающую информирование клиентов о конно-спортивных услугах, предварительную запись с учетом доступности ресурсов конюшни и демонстрационный режим администрирования контента.

Для достижения цели необходимо решить следующие задачи:

- проанализировать предметную область и действующий порядок взаимодействия клиента с предприятием;
- сформировать требования к клиентской части web-приложения и пользовательским сценариям;
- обосновать выбор технологий React, TypeScript, Vite и React Router;
- спроектировать структуру страниц, компонентов, данных и mock API layer;
- реализовать каталог услуг, карточку услуги, галерею, отзывы, контакты, правила безопасности и форму предварительной записи;
- разработать календарное бронирование с учетом лошадей, ограничений по весу, перерывов после занятий и нескольких участников;
- реализовать демонстрационный административный режим для редактирования контента, услуг, фотографий, лошадей, заявок и правил записи;
- проверить корректность сборки, адаптивность интерфейса и отсутствие критических ошибок пользовательского сценария.

Срок выполнения работы: 2026 год.

ЛИСТ ЗАМЕЧАНИЙ

Лист предназначен для замечаний руководителя, консультантов, нормоконтролера и членов государственной экзаменационной комиссии.

Замечания:

АННОТАЦИЯ

В выпускной квалификационной работе разработана клиентская часть web-приложения для малого предприятия сферы предоставления конно-спортивных услуг - ИП Орлова Н. И., г. Гурьевск. В работе рассмотрены особенности предметной области, связанные с необходимостью предварительной записи, правилами безопасности, ограниченным количеством лошадей, требованиями к возрасту, уровню подготовки и весу участников.

Результатом работы является адаптивное web-приложение на React и TypeScript, включающее главную страницу, каталог услуг, подробные карточки услуг, календарную форму предварительной записи, раздел правил и безопасности, галерею, отзывы, контакты и демонстрационный административный режим. В проекте подготовлены TypeScript-модели, mock-данные, слой API-запросов и frontend-логика проверки доступности временных слотов.

Практическая значимость работы заключается в создании единого клиентского интерфейса, который может быть интегрирован с backend API и использоваться как основа для внедрения цифрового канала взаимодействия предприятия с клиентами.

Ключевые слова: web-приложение, React, TypeScript, Vite, конно-спортивные услуги, предварительная запись, календарное бронирование, mock API, адаптивный интерфейс.

Орлов Д. С.

ANNOTATION

The final qualifying work presents the development of the client side of a web application for a small enterprise providing equestrian and horse-related leisure services: individual entrepreneur Orlova N. I., Guryevsk. The work considers the domain specifics: preliminary requests, safety rules, a limited number of horses, rider age, experience and weight restrictions, and the need to confirm bookings by an administrator.

The developed result is a responsive React and TypeScript web application that includes a home page, service catalog, service details, calendar-based booking form, safety rules, gallery, reviews, contacts and a demonstration administrative mode. The project contains typed data models, mock data, a replaceable API layer and frontend availability-checking logic for booking slots.

The practical value of the work is the creation of a unified client interface that can later be connected to a backend API and used as a basis for implementing a digital communication channel between the enterprise and its clients.

Keywords: web application, React, TypeScript, Vite, equestrian services, preliminary request, calendar booking, mock API, responsive interface.

Orlov D. S.

СОДЕРЖАНИЕ

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ	2
ЛИСТ ЗАМЕЧАНИЙ	3
АННОТАЦИЯ.....	4
ANNOTATION.....	5
СОДЕРЖАНИЕ	6
ВВЕДЕНИЕ	9
1 АНАЛИТИЧЕСКАЯ ЧАСТЬ.....	10
1.1 Характеристика предприятия и предметной области.....	10
1.2 Анализ действующего порядка взаимодействия с клиентами	11
1.3 Пользовательские требования.....	12
1.4 Выбор технологического стека	13
1.5 Обзор подходов к цифровизации малых предприятий сферы услуг	14
1.6 Теоретические основы построения клиентской части web-приложения	16
1.7 Теоретические требования к пользовательскому интерфейсу	17
1.8 Особенности предметной области, влияющие на алгоритм записи	19
1.9 Требования к безопасности и обработке персональных данных	20
1.10 Выводы по теоретической части	21
1.11 Анализ аналогичных пользовательских решений	22
1.12 Теоретическое обоснование требований к календарному бронированию.....	23
1.13 Методические выводы для дальнейшего проектирования	25
1.14 Теоретические основы проектирования данных frontend-приложения.....	25
1.15 Теоретическое обоснование выбора архитектуры без избыточного усложнения.....	27
1.16 Теоретические критерии оценки результата frontend-разработки	28

1.17 Роль прототипирования в разработке клиентской части	29
2 ПРОЕКТНАЯ ЧАСТЬ	30
2.1 Назначение и структура клиентской части.....	30
2.2 Модели данных TypeScript.....	31
2.3 API-контракт и mock API layer	32
2.4 Логика календарного бронирования	33
3 РЕАЛИЗАЦИЯ WEB-ПРИЛОЖЕНИЯ	34
3.1 Структура проекта.....	34
3.2 Реализация публичных страниц.....	35
3.3 Реализация формы предварительной записи.....	38
3.4 Реализация административного режима.....	39
3.5 Адаптивность и пользовательский интерфейс.....	41
3.6 Проверка работоспособности.....	42
3.7 Подробное техническое задание на клиентскую часть	42
3.8 Проектирование пользовательских сценариев.....	44
3.9 Реализация слоя данных и mock API.....	45
3.10 Реализация алгоритма доступности	46
3.11 Реализация формы записи и валидации.....	47
3.12 Реализация административного редактирования контента	49
3.13 Реализация публичных разделов и визуальной структуры	50
3.14 Проверка адаптивности и качества интерфейса	52
3.15 Подготовка проекта к backend-интеграции	53
3.16 Ограничения текущей реализации и направления развития	54
3.17 Информационное обеспечение разработанного приложения	55
3.18 Программное обеспечение и структура компонентов.....	56

3.19 Алгоритмическое обеспечение и обработка исключений	57
3.20 Организационное обеспечение и роли пользователей	58
3.21 Детализация тестовых сценариев	59
3.22 Эксплуатационные требования к будущей версии.....	61
3.23 Подробное описание реализации API-контракта.....	62
3.24 Детализация реализации административных вкладок	63
3.25 Подробное описание визуальной адаптации макетов	64
3.26 Контроль соответствия требованиям ВКР.....	65
3.27 Практическая проверка демонстрационного прототипа.....	66
3.28 Практическое значение реализованной клиентской части	67
3.29 Резерв практического сопровождения и передачи проекта.....	68
4 ОЦЕНКА РЕЗУЛЬТАТА И ТРЕБОВАНИЯ К ИНТЕГРАЦИИ	69
4.1 Ожидаемый эффект внедрения	69
4.2 Защита персональных данных и ограничения frontend-проверок.....	69
4.3 Согласование с backend-разработчиком	70
ЗАКЛЮЧЕНИЕ	70
СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ	72
ПРИЛОЖЕНИЕ А	74
ПРИЛОЖЕНИЕ Б.....	75
ПРИЛОЖЕНИЕ В.....	76

ВВЕДЕНИЕ

Малые предприятия сферы услуг все чаще используют web-приложения как основной канал первичного взаимодействия с клиентами. Для предприятия, предоставляющего конно-спортивные услуги, сайт выполняет не только информационную функцию, но и помогает снизить количество уточняющих обращений: клиент заранее видит виды занятий, стоимость, ограничения, правила поведения рядом с лошадьми и порядок предварительной записи.

Актуальность выбранной темы определяется тем, что деятельность конюшни имеет ряд ограничений, которые невозможно корректно учесть простой формой обратной связи. У предприятия ограничено количество лошадей и инструкторов, лошадям необходим отдых после занятий, разные услуги имеют различную длительность, а для участников учитываются возраст, уровень подготовки и вес. Поэтому клиентская часть должна не только показывать каталог услуг, но и подготавливать структурированную заявку для дальнейшей проверки администратором.

Проблема разработки состоит в необходимости объединить в одном интерфейсе информирование клиента, предварительную запись, правила безопасности, галерею, отзывы, контакты и демонстрационные средства управления содержанием, не реализуя при этом полноценный backend и базу данных. Такая постановка задачи соответствует распределению работ в команде: клиентская часть должна быть готова к интеграции с сервером, но на этапе ВКР работает с mock-данными.

Объектом исследования является процесс клиентского взаимодействия малого предприятия сферы предоставления конно-спортивных услуг. Предметом исследования является клиентская часть web-приложения, обеспечивающая просмотр информации, выбор услуги, календарное оформление предварительной заявки и редактирование контента в демонстрационном административном режиме.

Цель выпускной квалификационной работы - разработать клиентскую часть web-приложения для ИП Орлова Н. И., позволяющую клиенту изучить услуги предприятия и отправить предварительную заявку на занятие с учетом ограниченных ресурсов конюшни.

Для достижения цели в работе решаются задачи анализа предметной области, формирования требований, проектирования архитектуры frontend-приложения, реализации страниц и компонентов, разработки mock API layer, реализации календарного бронирования, создания демонстрационного администрирования и проверки качества полученного результата.

Методами работы являются анализ предметной области, структурное проектирование, компонентное проектирование пользовательского интерфейса, типизация данных средствами TypeScript, моделирование API-контракта, ручное функциональное тестирование и проверка адаптивности интерфейса. При оформлении работы учитываются требования ГОСТ 7.32 и методические указания по подготовке ВКР [1], [2].

Практическая значимость заключается в том, что разработанная клиентская часть может быть использована как демонстрационный прототип и как основа для дальнейшей интеграции с backend API. Проект показывает полный клиентский сценарий: от знакомства с предприятием до отправки заявки, а также демонстрирует администраторский режим для управления услугами, фотографиями, лошадьми, заявками и правилами записи.

1 АНАЛИТИЧЕСКАЯ ЧАСТЬ

1.1 Характеристика предприятия и предметной области

ИП Орлова Н. И. осуществляет деятельность в сфере предоставления конно-спортивных услуг в г. Гурьевске. Предприятие ориентировано на клиентов, которые интересуются обучением верховой езде, прогулками, индивидуальными занятиями, фотосессиями, досуговыми мероприятиями и безопасным общением с лошадьми. В

отличие от типового сервиса записи на услугу, деятельность конюшни связана с живыми ресурсами, поэтому расписание требует учета состояния лошадей, перерывов, допустимых нагрузок и подготовки участников.

Основные сведения о предприятии представлены в таблице 1. Данные сформированы на основе материалов преддипломной практики и используются как исходная информация для проектирования клиентской части.

Таблица 1 - Общая характеристика ИП Орлова Н. И.

Показатель	Характеристика
Наименование	ИП Орлова Н. И.
Местонахождение	г. Гурьевск
Сфера деятельности	Предоставление конно-спортивных услуг
Основные услуги	Обучение верховой езде, прогулки, индивидуальные занятия, фотосессии, досуговые мероприятия
Целевая аудитория	Дети и родители, взрослые начинающие клиенты, клиенты с опытом верховой езды, гости города, семьи
Особенности процесса	Предварительная запись, учет безопасности, состояния лошадей, погодных условий, возраста и веса участников
Назначение web-приложения	Единый канал информирования, выбора услуги и отправки предварительной заявки

1.2 Анализ действующего порядка взаимодействия с клиентами

До внедрения web-приложения клиентское взаимодействие малого предприятия может быть распределено между несколькими каналами: личные обращения, телефон, мессенджеры, социальные сети и устные рекомендации. Такой формат позволяет быстро отвечать на отдельные вопросы, но не формирует единую структуру данных об услугах, заявках, расписании и правилах посещения.

Для клиента это выражается в необходимости уточнять стоимость, длительность, возрастные ограничения, требования к одежде и порядок записи. Для администратора возникает дополнительная нагрузка, поскольку требуется ручную

проверять свободное время, состояние лошадей, допустимость услуги и возможность принять несколько участников одновременно. На рисунке 1 представлена контекстная схема процесса оказания услуги.

Клиент	->	Выбор услуги и отправка заявки	->	Администратор
Правила безопасности	->	Проверка условий участия	->	Лошади и инструкторы
Расписание	->	Подтверждение заявки	->	Оказание услуги

Рисунок 1 - Контекстная схема процесса оказания конно-спортивной услуги

В ходе анализа выделены проблемы, которые должны быть устранены средствами клиентской части и дальнейшей серверной интеграции. Эти проблемы и проектные решения приведены в таблице 2.

Таблица 2 - Проблемы действующего процесса и способы их устранения

Проблема	Проявление	Решение в web-приложении
Разрозненность информации	Сведения об услугах и правилах находятся в разных каналах	Единый каталог услуг, страницы правил, контактов, отзывов и галереи
Ручная обработка заявок	Администратор уточняет данные клиента и параметры занятия в переписке	Структурированная форма предварительной записи
Ограниченные ресурсы конюшни	Нельзя записать клиента на любое время без учета лошадей и перерывов	Календарное бронирование с mosk-проверкой доступности
Необходимость учитывать нескольких участников	Родитель или организатор может записывать группу	Список участников с отдельными параметрами возраста, веса и опыта
Сложность актуализации контента	Фото, цены и описания требуют оперативных изменений	Демонстрационный административный режим с редактированием контента
Мобильный сценарий	Клиенты часто открывают сайт со смартфона	Mobile-first верстка, крупные кнопки и адаптивная навигация

1.3 Пользовательские требования

Клиентская часть должна быть понятной для широкой аудитории, включая людей, впервые посещающих конюшню. Поэтому интерфейс использует русский язык, избегает технических терминов и показывает важные ограничения заранее. Для родителей несовершеннолетних участников особенно важны правила безопасности, возрастные ограничения и возможность указать индивидуальные особенности ребенка.

Таблица 3 - Пользовательские требования к клиентской части

Пользователь	Цель	Требование к интерфейсу
Новый клиент	Понять, чем занимается предприятие	Главная страница с кратким описанием, преимуществами, отзывами и быстрыми переходами
Клиент, выбирающий услугу	Сравнить форматы занятий	Каталог с карточками, ценами, длительностью и ограничениями
Клиент, оформляющий заявку	Передать данные для предварительной записи	Форма с выбором услуги, даты, времени, участников и согласием на обработку данных
Родитель или организатор группы	Записать нескольких участников	Добавление и удаление участников, отдельные поля возраста, веса и опыта
Администратор	Обновлять сайт и управлять заявками	Режим администрирования с редактированием фото, текста, услуг, лошадей, календаря и правил
Backend-разработчик	Подключить серверную часть	Выделенный API layer, TypeScript-типы и mock-функции, повторяющие будущие endpoints

1.4 Выбор технологического стека

Для реализации клиентской части выбран стек React, TypeScript, Vite и React Router. React обеспечивает компонентный подход к построению интерфейса, TypeScript позволяет формализовать модели данных и снизить риск ошибок при интеграции с API, Vite ускоряет разработку и сборку, а React Router организует маршруты публичных страниц и административного раздела [7]-[10].

Таблица 4 - Используемые технологии

Технология	Назначение в проекте	Обоснование выбора
React	Построение интерфейса из компонентов	Поддерживает повторное использование карточек, форм, списков и административных блоков
TypeScript	Типизация данных и API-контракта	Позволяет описать Service, BookingRequest, Horse, BookingRule и другие модели
Vite	Среда разработки и сборки	Быстрый dev-сервер, простая конфигурация, поддержка React
React Router	Маршрутизация страниц	Позволяет реализовать каталог, карточку услуги, запись, правила, галерею, контакты и админку
CSS	Адаптивная верстка и оформление	Достаточно для дипломного frontend-проекта без усложнения архитектуры
Fetch/mock API layer	Подготовка к backend-интеграции	Компоненты не зависят напрямую от mock-данных

1.5 Обзор подходов к цифровизации малых предприятий сферы услуг

Для малого предприятия web-приложение является не только средством рекламы, но и элементом информационной системы, который формализует порядок взаимодействия с клиентами. Если сайт ограничивается статичной визиткой, он решает только задачу первичного знакомства. В рассматриваемом проекте требуется более широкий подход: интерфейс должен поддерживать выбор услуги, предварительную запись, передачу данных в структурированном виде и дальнейшее сопровождение заявки администратором.

В сфере услуг цифровой канал особенно важен из-за высокой доли повторяющихся вопросов. Клиент уточняет состав услуги, стоимость, продолжительность, возрастные ограничения, требования к подготовке, возможность посещения группой и порядок отмены или переноса занятия. При отсутствии единого интерфейса такие вопросы обрабатываются в разных каналах связи, что приводит к дублированию ответов, потере контекста и увеличению нагрузки на сотрудника.

Web-приложение для малого предприятия должно быть проще корпоративной информационной системы, но сложнее рекламной страницы. Оно должно давать клиенту возможность выполнить основные действия без прямого участия

администратора, а администратору - получать данные в виде, пригодном для дальнейшей обработки. Такой подход соответствует постепенной цифровизации: сначала внедряется понятный клиентский интерфейс и API-контракт, затем подключаются сервер, база данных, уведомления и аналитика.

При проектировании решения для ИП Орлова Н. И. учитывается, что предприятие не является крупным спортивным комплексом с большим административным штатом. Поэтому интерфейс должен быть поддерживаемым, а структура данных - достаточно простой. Избыточное усложнение архитектуры привело бы к тому, что проект было бы трудно сопровождать после защиты. В то же время отсутствие разделения данных, компонентов и сервисов затруднило бы интеграцию с backend-разработчиком.

Сравнение типовых решений показывает, что для малого бизнеса часто используются конструкторы сайтов, страницы в социальных сетях, формы обратной связи и мессенджеры. Их преимущество заключается в быстром запуске, но недостаток состоит в слабой формализации бизнес-логики. Конструктор может показать услуги, однако не учитывает доступность конкретных лошадей, ограничения по весу, перерывы после занятий и несколько участников в одной заявке. Поэтому для данного проекта оправдана разработка собственного frontend-приложения.

Другой вариант - подключение готового сервиса онлайн-записи. Такие сервисы хорошо подходят для салонов, консультаций или приема специалистов, где ресурсом является один сотрудник или кабинет. Для конюшни модель сложнее: один временной слот может требовать нескольких лошадей, каждая лошадь имеет допустимые услуги и ограничения по весу, а после занятия ей требуется отдых. Следовательно, обычная таблица свободного времени не отражает реальные ограничения предметной области.

Разрабатываемое приложение занимает промежуточное положение между универсальной CRM-системой и простым сайтом. Оно не реализует полноценную учетную систему, но подготавливает интерфейс и модели данных к будущему

развитию. Такой подход соответствует задачам выпускной квалификационной работы: студент выполняет клиентскую часть, а backend, база данных и окончательная обработка заявок будут реализованы другим участником команды.

1.6 Теоретические основы построения клиентской части web-приложения

Клиентская часть современного web-приложения отвечает за отображение данных, пользовательский ввод, локальную проверку формы, навигацию и взаимодействие с сервером. В отличие от статического сайта, клиентское приложение хранит часть состояния на стороне браузера: выбранную услугу, выбранную дату, список участников, состояние загрузки, ошибки и успешный результат отправки. Правильная организация этого состояния влияет на удобство интерфейса и надежность пользовательского сценария.

Компонентный подход предполагает разбиение интерфейса на независимые блоки: кнопки, карточки, формы, списки, секции, сообщения состояния, элементы FAQ, карточки отзывов и административные редакторы. Преимущество такого подхода состоит в повторном использовании элементов и уменьшении дублирования кода. Например, одна и та же кнопка может использоваться на главной странице, в карточке услуги, в форме записи и в административном разделе, сохраняя единый визуальный стиль.

Для дипломного проекта важно не только реализовать визуальные страницы, но и показать архитектурное разделение ответственности. Компонент не должен напрямую знать, где хранятся услуги или правила записи. Он должен вызвать функцию сервисного слоя и получить данные в ожидаемом формате. Благодаря этому mock-данные можно заменить на реальные HTTP-запросы без переписывания всех страниц. Такой принцип соответствует практике построения поддерживаемых frontend-приложений.

TypeScript используется для явного описания структуры данных. В проекте типами описаны услуги, заявки, участники, лошади, правила бронирования, временные слоты, отзывы, элементы галереи и контактная информация. Это снижает

вероятность ошибок при передаче данных между компонентами и сервисами. Кроме того, типы служат документацией для backend-разработчика: по ним видно, какие поля должен возвращать API и какие данные frontend отправляет на сервер.

Маршрутизация необходима, чтобы пользователь мог напрямую открыть нужную страницу, вернуться к каталогу, перейти из карточки услуги в форму записи и получить понятную структуру адресов. React Router позволяет связать адреса с компонентами страниц. Для пользователя это выражается в привычной навигации, а для разработчика - в четком разделении публичных страниц и административного режима.

Состояния loading, error и success являются обязательными для интерфейса, который работает с данными. Даже если в учебной версии данные возвращаются из mock API, структура приложения должна учитывать задержки сети, ошибки сервера и успешные ответы. Это позволяет заранее продумать текст сообщений и не показывать пользователю технические детали. Например, при ошибке отправки заявки клиент должен увидеть понятную фразу о необходимости повторить попытку или связаться с администратором.

Особенность frontend-части заключается в том, что она работает в недоверенной среде: пользователь может изменить данные в браузере, отключить JavaScript, подменить запрос или отправить некорректные значения напрямую на сервер. Поэтому клиентская проверка формы рассматривается только как средство удобства, а не как окончательная защита. В рабочей версии backend обязан повторять все проверки доступности, прав доступа и корректности данных.

1.7 Теоретические требования к пользовательскому интерфейсу

Пользовательский интерфейс для конно-спортивных услуг должен учитывать неоднородность аудитории. Сайт могут открывать родители, начинающие взрослые клиенты, уверенные наездники, гости города и люди, выбирающие фотосессию или семейный досуг. Часть пользователей уже понимает специфику занятий с лошадьми, а часть впервые сталкивается с требованиями безопасности. Поэтому интерфейс

должен быть спокойным, ясным и не перегруженным профессиональными терминами.

Главная страница должна выполнять роль ориентира. Пользователь за первые секунды должен понять, что предприятие предлагает обучение, прогулки, индивидуальные занятия и мероприятия с лошадьми. Кнопки «Посмотреть услуги» и «Записаться» должны быть заметными, поскольку они соответствуют двум основным сценариям: выбору формата занятия и оформлению заявки. При этом главная страница не должна превращаться в рекламный лендинг без функциональности, так как цель проекта шире обычной презентации.

Каталог услуг должен помогать сравнению. Для этого в карточках показываются название, краткое описание, длительность, цена, ограничения и доступность. Если клиенту нужно открыть подробности, он переходит в карточку услуги. Если решение уже принято, он может сразу перейти к записи. Такая структура сокращает путь постоянного клиента и одновременно дает новому клиенту возможность изучить детали.

Форма записи должна быть построена по принципу постепенного раскрытия информации. Сначала выбирается услуга и дата, затем временной слот, затем участники и контактные данные. Если показать все поля без логики, клиент может ошибиться или не понять, почему определенное время недоступно. Календарный сценарий помогает связать выбор даты и времени с реальными ограничениями конюшни.

Отображение причин недоступности является важным элементом доверия. Сообщение «слот недоступен» не объясняет ситуацию и может восприниматься как ошибка сайта. Более полезны причины: «нет свободных лошадей», «для участника нет подходящей лошади по весу», «день закрыт для записи», «лошадь должна отдохнуть после предыдущей тренировки». Такие формулировки не раскрывают лишних внутренних данных, но помогают клиенту выбрать другой слот.

Адаптивность интерфейса особенно важна для малого предприятия, так как значительная часть обращений формируется с мобильных устройств. На смартфоне карточки должны располагаться в одну колонку, поля формы должны быть крупными, а кнопки - доступными для нажатия пальцем. При этом на desktop-версии нужно эффективно использовать ширину экрана, показывая сетки услуг, галереи и административные формы без лишней прокрутки.

Доступность интерфейса включает не только соответствие стандартам WCAG, но и понятные подписи полей, достаточную контрастность, отсутствие текста только внутри изображений, корректные состояния кнопок и возможность прочесть ошибки формы. Для проекта конюшни это важно еще и потому, что интерфейс должен быть понятен людям разного возраста и уровня технической подготовки.

1.8 Особенности предметной области, влияющие на алгоритм записи

Конно-спортивная услуга отличается от большинства бытовых услуг тем, что ресурсом является не только сотрудник, но и животное. Лошадь не может быть назначена на занятия без учета состояния, нагрузки и перерыва после предыдущей тренировки. Поэтому алгоритм записи должен рассматривать лошадь как ограниченный ресурс с собственными характеристиками: допустимый вес наездника, список разрешенных услуг, статус доступности и индивидуальные заметки администратора.

Вес наездника является критичным параметром безопасности. Если форма записи не собирает вес участника, администратор вынужден уточнять его вручную, а система не может даже предварительно оценить доступность слота. В проекте вес вводится для каждого участника, что позволяет выполнить mock-подбор лошадей и заранее показать, что выбранный слот может быть недоступен.

Возраст участника и уровень подготовки также влияют на выбор услуги и организацию занятия. Для новичков может потребоваться более спокойный формат и дополнительное сопровождение инструктора. Для детей важны отдельные требования к безопасности и согласию родителей. Поэтому модель участника

включает возраст, опыт и комментарий, где клиент может указать индивидуальные особенности.

Несколько участников в одной заявке усложняют проверку доступности. Если один клиент записывает двух детей или группу друзей, система должна проверить наличие подходящей лошади для каждого участника. Недостаточно убедиться, что есть хотя бы одна свободная лошадь. Нужно подобрать набор ресурсов, не назначая одну и ту же лошадь двум участникам на один слот.

Перерыв между тренировками является правилом, которое защищает ресурс и поддерживает качество услуги. Если занятие заканчивается в 15:00, то при правиле отдыха 30 минут лошадь не должна быть доступна для следующего занятия в 15:00. Следующий возможный слот для этой лошади начинается не раньше 15:30. В интерфейсе это правило выражается в недоступности отдельных слотов или уменьшении количества подходящих лошадей.

Погодные условия и сезонность также могут влиять на запись. В рамках frontend-проекта они представлены через правила закрытых дат и исключений расписания. В будущей версии backend может расширить эту модель, добавив автоматическое закрытие дней, переносы занятий и уведомления клиентов. Клиентская часть уже подготовлена к отображению причин недоступности и статусов заявок.

1.9 Требования к безопасности и обработке персональных данных

Форма предварительной записи обрабатывает персональные данные: имя клиента, номер телефона, имя участника, возраст, вес и комментарии. В соответствии с требованиями к обработке персональных данных пользователь должен явно подтвердить согласие перед отправкой заявки [4]. В интерфейсе это реализуется через обязательный чекбокс, без которого отправка невозможна.

На клиентской стороне не следует хранить персональные данные в localStorage без необходимости. Такой подход снижает риск случайного сохранения контактной информации на чужом устройстве. В текущем проекте редактируемый

административный контент может сохраняться локально для демонстрации, но данные реальных клиентов должны передаваться на сервер и храниться в защищенной базе данных в соответствии с политикой предприятия.

Административный вход в учебной версии является демонстрационным. Он показывает будущий сценарий управления контентом, но не должен рассматриваться как полноценная защита. В промышленной версии требуется серверная авторизация, разграничение ролей, защита endpoint, проверка сессии, журналирование действий администратора и ограничение доступа к персональным данным.

Безопасность frontend-приложения включает также корректную обработку ошибок. Пользователю не следует показывать стек вызовов, внутренние идентификаторы или технические сообщения сервера. Вместо этого интерфейс должен выводить понятный текст и, при необходимости, предложить связаться с администратором. Для администратора технические сведения могут быть доступны в отдельном защищенном журнале, который выходит за рамки текущей клиентской реализации.

При дальнейшей интеграции с backend необходимо использовать HTTPS, проверять данные на сервере, ограничивать размер загружаемых изображений, проверять типы файлов, защищать API от несанкционированных изменений и учитывать требования OWASP. Эти меры не отменяют удобную клиентскую проверку, но делают систему устойчивой к намеренной или случайной передаче некорректных данных.

1.10 Выводы по теоретической части

Проведенный анализ показывает, что для конно-спортивного предприятия требуется web-приложение, учитывающее не только презентацию услуг, но и особенности предварительной записи. Простая форма обратной связи не отражает ограниченность ресурсов, необходимость отдыха лошадей, весовые ограничения и возможность записи нескольких участников. Поэтому клиентская часть должна быть спроектирована как функциональный интерфейс, готовый к интеграции с backend.

Теоретический анализ подтвердил целесообразность компонентной архитектуры, выделения API layer, использования TypeScript-моделей и разделения бизнес-логики доступности от React-компонентов. Эти решения позволяют сохранить понятность учебного проекта и одновременно показать профессиональный подход к разработке клиентской части.

На основе анализа предметной области сформированы требования к страницам, форме записи, административному режиму, обработке состояний и безопасности. Эти требования далее используются в проектной и практической части работы при описании архитектуры, алгоритмов, модулей и результатов реализации.

1.11 Анализ аналогичных пользовательских решений

Для обоснования проектных решений рассмотрены типовые категории решений, которые могут использоваться малым предприятием для представления услуг и приема обращений. К ним относятся страницы в социальных сетях, сайты-визитки, конструкторы сайтов, универсальные сервисы записи, CRM-системы и индивидуально разработанные web-приложения. Каждая категория решает часть задач, однако не всегда учитывает специфику конно-спортивной деятельности.

Страница в социальной сети позволяет быстро публиковать новости, фотографии и отзывы. Такой канал удобен для продвижения, но не обеспечивает структурированный каталог услуг и формализованную заявку. Клиенту приходится просматривать публикации, искать актуальную цену, писать в сообщения и ждать ответа. Для администратора такой поток обращений труднее анализировать, потому что данные остаются внутри переписки.

Сайт-визитка решает проблему представления основной информации, но обычно не содержит сложной логики выбора времени. Он может показать адрес, телефон, перечень услуг и фотографии, однако форма обратной связи часто передает только имя, телефон и комментарий. Для конюшни этого недостаточно, поскольку нужно учитывать участников, вес, опыт, длительность услуги и свободные ресурсы.

Конструкторы сайтов позволяют без программирования создать страницу с блоками, формами и галереями. Их преимущество - низкий порог запуска. Недостаток - ограниченная возможность описать предметную бизнес-логику. Встроенная форма может отправить заявку, но не проверит, есть ли подходящая лошадь для участника весом 82 кг и не занята ли она с учетом перерыва после тренировки.

Универсальные сервисы онлайн-записи ориентированы на типовые услуги: прием специалиста, аренду кабинета, консультацию, занятие с одним преподавателем. Они хорошо работают там, где ресурс можно представить как одного сотрудника и временной слот. В случае конюшни ресурсная модель сложнее: один заказ может требовать несколько лошадей, а каждая лошадь имеет собственные ограничения и статусы.

CRM-система может быть полезна для учета клиентов, заявок и истории взаимодействий, но ее внедрение часто требует настройки, обучения сотрудников и финансовых затрат. Для выпускной квалификационной работы задача состоит не во внедрении готовой CRM, а в разработке клиентской части, которая может стать внешним интерфейсом будущей информационной системы предприятия.

Индивидуальная frontend-разработка позволяет точно отразить сценарии предприятия. В проекте можно сразу заложить сущности Service, BookingRequest, Horse, BookingRule и TimeSlot, а также подготовить API-контракт. Такой подход требует больше разработки на первом этапе, но дает гибкость и демонстрирует профессиональный уровень проектирования.

По результатам сравнения выбран вариант индивидуального web-приложения на React и TypeScript. Он обеспечивает баланс между демонстрационной реализуемостью и готовностью к расширению. В текущей версии приложение работает на mock-данных, но структура файлов и сервисов позволяет заменить mock API реальным backend без изменения пользовательского сценария.

1.12 Теоретическое обоснование требований к календарному бронированию

Календарное бронирование в предметной области конюшни является задачей распределения ограниченных ресурсов во времени. Ресурсами выступают лошади, инструкторы и доступные интервалы расписания. В текущей клиентской части моделируется прежде всего ресурс лошади, так как именно он определяет ключевые ограничения: допустимую нагрузку, вес наездника, статус здоровья, занятость и необходимость отдыха.

В отличие от обычной формы записи, календарное бронирование требует динамического обновления интерфейса. Пользователь выбирает услугу, после чего список дат и слотов зависит от длительности услуги и правил работы. Затем пользователь добавляет участников, и доступность слотов может измениться, потому что для каждого участника нужно найти подходящую лошадь. Это означает, что форма не может быть полностью статической.

С точки зрения теории проектирования интерфейса важно не перегружать клиента внутренней логикой. Пользователю не нужно видеть полный алгоритм распределения лошадей, но он должен понимать результат. Поэтому интерфейс показывает доступные и недоступные слоты, а для недоступных слотов выводит короткую причину. Такая обратная связь помогает пользователю скорректировать выбор, не обращаясь к администратору по базовым вопросам.

При наличии нескольких участников задача усложняется до подбора набора ресурсов. Если для первого участника подходит легкая лошадь, а для второго - только одна более сильная лошадь, алгоритм должен избежать неудачного назначения. В учебной версии используется упрощенный жадный подход, но сама модель данных позволяет backend-разработчику заменить его более точным алгоритмом без изменения формы.

Правило отдыха после занятия является примером временного ограничения. Оно показывает, что занятость ресурса не заканчивается строго в момент окончания услуги. Для лошади интервал блокировки включает само занятие и дополнительный период восстановления. Такая модель полезна и для будущих правил: уборка манежа,

подготовка оборудования, перерыв инструктора или закрытие территории на мероприятие.

Таким образом, календарное бронирование является центральной бизнес-функцией проекта. Оно превращает сайт из информационной витрины в прикладной инструмент, который учитывает реальную специфику конюшни. Именно поэтому логика доступности вынесена в отдельный сервис, а не размещена внутри компонента формы.

1.13 Методические выводы для дальнейшего проектирования

Методические материалы требуют, чтобы специальная часть ВКР содержала не только скриншоты разработанной системы, но и схемы, структуры, алгоритмы и виды обеспечения. Поэтому дальнейшая часть работы строится вокруг нескольких уровней описания: пользовательские сценарии, информационное обеспечение, программное обеспечение, алгоритмическое обеспечение, интерфейсные решения, проверка качества и требования к интеграции.

Для данной темы важны информационное и алгоритмическое обеспечение. Информационное обеспечение включает структуру данных об услугах, клиентах, участниках, лошадях, заявках, правилах и фотографиях. Алгоритмическое обеспечение включает расчет доступных дат, проверку временных слотов, подбор лошадей, проверку конфликтов и формирование причин недоступности.

Программное обеспечение раскрывается через структуру React-проекта, компоненты, маршруты, сервисы и стили. Организационное обеспечение раскрывается через будущий процесс работы администратора: управление контентом, обработка заявок, подтверждение или отклонение, изменение расписания и актуализация правил. Все эти аспекты включены в практическую часть, чтобы работа соответствовала не только программной реализации, но и логике внедрения.

1.14 Теоретические основы проектирования данных frontend-приложения

Проектирование данных в клиентской части начинается с определения устойчивых сущностей предметной области. Для сайта конюшни такими сущностями являются услуга, заявка, участник, лошадь, правило записи, временной слот, отзыв, элемент галереи и контактная информация. Если эти сущности не выделить заранее, компоненты начнут использовать разнородные объекты, что усложнит поддержку и интеграцию с backend.

В frontend-разработке данные часто проходят через несколько уровней: источник данных, сервисный слой, состояние страницы и отображающий компонент. На каждом уровне важно сохранять понятную структуру. Например, услуга в mosk-данных содержит длительность в минутах, ограничения, правила безопасности и изображение. Сервис возвращает эту услугу в ApiResponse, страница получает ее по id, а компонент карточки отображает отдельные поля.

При проектировании модели BookingRequest важно отделить данные клиента от данных участников. Клиент может быть родителем или организатором группы, а участники фактически будут наездниками. Поэтому имя и телефон клиента не должны подменять имена участников. Такая модель лучше соответствует реальному процессу и позволяет администратору корректно подготовить занятие.

Модель Horse отражает ресурсную сущность. В отличие от обычного справочника, эта модель влияет на доступность записи. Поля maxRiderWeightKg, allowedServiceIds, status и isActive участвуют в алгоритме. Это означает, что изменение данных лошади является не только изменением карточки, но и изменением правил планирования.

Модель BookingRule выбрана достаточно гибкой: тип правила задается строковым union-типом, а конфигурация хранится как Record<string, unknown>. Такой подход допускает разные виды правил без немедленного создания отдельной таблицы под каждое правило. Для учебного frontend-проекта это разумный компромисс между типизацией и расширяемостью.

Для будущей серверной реализации важно согласовать, какие поля являются обязательными, какие могут отсутствовать, какие значения допустимы для статусов и как кодируются даты. Если frontend и backend используют разные представления даты или статуса, ошибки появятся не в момент компиляции, а при выполнении. TypeScript-типы помогают обнаружить такие несоответствия раньше.

Отдельное значение имеет модель ApiResponse. Она позволяет возвращать не просто данные, но и сообщение. В будущем в этот объект можно добавить код ошибки, список валидационных ошибок или метаданные пагинации. Наличие общей обертки делает ответы API более единообразными и упрощает обработку состояний на страницах.

1.15 Теоретическое обоснование выбора архитектуры без избыточного усложнения

Архитектура дипломного frontend-проекта должна быть достаточной для демонстрации профессионального подхода, но не должна превращаться в избыточную корпоративную систему. Поэтому в работе не используются сложные state-management библиотеки, микрофронтенды или генерация API-клиента. Вместо этого применена понятная структура каталогов и отдельные сервисные функции.

Такой выбор соответствует масштабу задачи. Приложение содержит несколько публичных страниц, форму записи, mock API, административный режим и алгоритм доступности. Для этого достаточно React hooks, React Router, TypeScript и сервисных модулей. Добавление более тяжелых решений увеличило бы объем кода и усложнило бы защиту без существенной пользы для результата.

При этом архитектура не является примитивной. В ней выделены страницы, компоненты, типы, данные, сервисы и стили. Бизнес-логика доступности вынесена отдельно. Редактируемый контент хранится через отдельный сервис. Компоненты используют данные через свойства и функции, а не напрямую из mock-массивов. Это показывает, что проект готов к развитию.

Важным архитектурным решением является сохранение границы между публичной частью и административным режимом. Публичные страницы должны оставаться понятными для клиента, а административные элементы появляются только после входа. Даже если вход демонстрационный, само разделение режимов помогает показать будущую модель прав доступа.

Еще одно решение - отказ от полноценной базы данных на стороне frontend. Браузер не должен быть источником истины для расписания и персональных данных. Поэтому localStorage применяется только для демонстрационного редактирования контента. Это ограничение соответствует требованиям безопасности и готовит проект к серверной модели хранения.

Таким образом, выбранная архитектура является сбалансированной: она не усложняет учебный проект, но содержит все ключевые элементы реального frontend-приложения. Она показывает, как проект может быть расширен backend API, загрузкой файлов, серверной авторизацией и постоянным хранением данных.

1.16 Теоретические критерии оценки результата frontend-разработки

Результат frontend-разработки оценивается не только по внешнему виду страниц. Для дипломного проекта важно показать, что интерфейс решает задачу предметной области, имеет понятную структуру кода, учитывает пользовательские сценарии, готов к backend-интеграции и не противоречит требованиям безопасности. Поэтому критерии оценки должны включать функциональность, удобство, адаптивность, типизацию и сопровождаемость.

Функциональность проверяется по сценариям: просмотр услуг, открытие карточки, изучение правил, заполнение заявки, добавление участников, выбор даты и слота, получение сообщения о статусе, просмотр контактов и использование административного режима. Если какой-либо из этих сценариев не работает, приложение не достигает цели ВКР.

Удобство оценивается через ясность интерфейса. Пользователь должен понимать, какое действие доступно сейчас, почему слот недоступен, какие поля

обязательны и что произойдет после отправки. Особенно важно не создавать ложного впечатления автоматического подтверждения записи, так как в реальности заявка должна проверяться администратором.

Адаптивность проверяется на разных ширинах экрана. Поскольку клиенты малого предприятия часто используют смартфоны, мобильная версия не может быть второстепенной. Форма записи, галерея и контакты должны быть удобными именно на мобильном устройстве, а desktop-версия должна использовать пространство для более полного представления данных.

Сопровождаемость оценивается по тому, насколько легко заменить mock API, добавить услугу, изменить модель или расширить админку. Если данные жестко зашиты в компоненты, проект трудно развивать. В реализованной архитектуре данные и сервисы вынесены отдельно, что повышает сопровождаемость и делает результат пригодным для дальнейшей командной работы.

1.17 Роль прототипирования в разработке клиентской части

Прототипирование является важным этапом разработки клиентской части, потому что позволяет проверить логику страниц до появления полноценной серверной части. В рассматриваемом проекте прототипирование выражается не только в визуальных макетах, но и в работающем frontend-приложении с mock API. Такой прототип можно показать заказчику, руководителю ВКР и backend-разработчику, чтобы согласовать пользовательские сценарии и структуру данных.

Работающий прототип полезнее статичного макета, когда речь идет о календарном бронировании. Макет может показать форму, но не показывает, как меняются слоты при выборе даты, добавлении участника или изменении веса. Mock-логика позволяет продемонстрировать динамическое поведение и заранее выявить спорные моменты: какие причины недоступности понятны клиенту, сколько полей должно быть у участника, где размещать сообщение о подтверждении.

Для малого предприятия прототип также снижает риск неверного внедрения. До разработки backend можно понять, какие разделы действительно нужны, какие

данные должен менять администратор, какие фотографии важны для галереи и какие поля заявки являются обязательными. Это помогает избежать ситуации, когда серверная часть создается под непроверенный интерфейс.

В результате прототип в данной ВКР выполняет двойную функцию: он является готовой клиентской частью для демонстрации и одновременно техническим заданием для следующего этапа разработки. Это соответствует практической направленности работы и подтверждает значимость выделения mock-данных, типов и API layer.

2 ПРОЕКТНАЯ ЧАСТЬ

2.1 Назначение и структура клиентской части

Клиентская часть web-приложения выполняет роль интерфейсного слоя между пользователем и будущей серверной частью. В текущей реализации сервер и база данных не создаются: их функции имитируются mock-данными и сервисами. Такой подход позволяет показать полный пользовательский сценарий и подготовить проект к последующей интеграции.

Маршруты приложения приведены в таблице 5. Каждый маршрут связан с отдельной страницей React и использует общие компоненты Layout, Header, Footer, Button, SectionTitle, ServiceCard, BookingForm, GalleryGrid, ReviewCard и другие.

Таблица 5 - Маршруты web-приложения

Маршрут	Страница	Назначение
/	HomePage	Главная страница с описанием предприятия, услугами, доверием и отзывами
/services	ServicesPage	Каталог услуг
/services/:id	ServiceDetailsPage	Подробная карточка услуги
/booking	BookingPage	Календарная форма предварительной записи
/rules	RulesPage	Правила посещения и безопасность
/gallery	GalleryPage	Галерея с разделами и папками фотографий

/reviews	ReviewsPage	Отзывы клиентов
/contacts	ContactsPage	Адрес, телефон, режим обработки заявок и быстрые действия
/admin	AdminPage	Демонстрационный вход и режим управления контентом

Общая архитектура клиентской части показана на рисунке 2. Компоненты страниц обращаются не к mock-данным напрямую, а к сервисному слою, который возвращает Promise и имитирует задержку сетевого запроса.

React Router	->	Страницы	->	Компоненты UI
Страницы	->	services/mockApi.ts	->	data/mockData.ts
BookingForm	->	availabilityService.ts	->	Horses, Rules, Bookings
AdminPage	->	adminContent.ts	->	localStorage demo state
mockApi.ts	=>	Будущий backend API	=>	База данных

Рисунок 2 - Архитектура клиентской части web-приложения

2.2 Модели данных TypeScript

Типизация данных вынесена в файл src/types/index.ts. Это позволяет согласовать структуру frontend и backend заранее. Основные модели данных представлены в таблице 6.

Таблица 6 - Основные TypeScript-модели

Модель	Назначение	Ключевые поля
Service	Описание услуги	id, title, durationMinutes, price, ageLimit, preparation, restrictions, safetyRules
BookingRequest	Данные предварительной заявки клиента	clientName, clientPhone, serviceId, date, timeSlotId, participants, personalDataAgreement
BookingParticipant	Данные участника занятия	fullName, age, weightKg, experience, comment
Horse	Ресурс конюшни	name, maxRiderWeightKg, allowedServiceIds, status, isActive, notes
Booking	Заявка в календаре администратора	date, startTime, endTime, participants, assignedHorses, status

BookingRule	Правило доступности записи	type, isActive, config
TimeSlot	Временной слот для клиента	startTime, endTime, isAvailable, reasons, availableHorseIds
SiteContent	Редактируемое содержимое сайта	siteName, hero-тексты, цвета, изображение главной, тексты страниц

2.3 API-контракт и mock API layer

Файл `src/services/mockApi.ts` содержит функции, соответствующие будущим endpoint серверной части. Компоненты вызывают эти функции как асинхронные операции, поэтому при появлении backend достаточно заменить внутреннюю реализацию функций на реальные HTTP-запросы через Fetch API или Axios.

Таблица 7 - Проектируемые API endpoints

Frontend-функция	Будущий endpoint	Назначение
getServices()	GET /api/services	Получение списка услуг
getServiceById(id)	GET /api/services/{id}	Получение карточки услуги
createBookingRequest(data)	POST /api/bookings	Создание предварительной заявки
getHorses()	GET /api/horses	Получение списка лошадей
createHorse(data)	POST /api/horses	Добавление лошади
updateHorse(id, data)	PATCH /api/horses/{id}	Редактирование лошади
deleteHorse(id)	DELETE /api/horses/{id}	Удаление лошади
getBookings()	GET /api/bookings	Получение заявок для календаря
updateBookingStatus(id, status)	PATCH /api/bookings/{id}/status	Изменение статуса заявки
assignBookingHorses(id, data)	PATCH /api/bookings/{id}/assign-horses	Назначение лошадей участникам
getAvailability(serviceId, date)	GET /api/availability	Получение слотов на дату

checkAvailability(data)	POST /api/availability/check	Проверка заявки перед отправкой
getBookingRules()	GET /api/booking-rules	Получение правил записи
getGalleryItems()	GET /api/gallery	Получение фотографий галереи
getContacts()	GET /api/contacts	Получение контактных данных

2.4 Логика календарного бронирования

Особенность предметной области состоит в том, что свободное время зависит не только от рабочего графика, но и от доступности лошадей. Поэтому в проекте выделен файл `src/services/availabilityService.ts`. Компоненты не содержат бизнес-логику проверки слотов: они получают готовые даты, слоты, статусы и причины недоступности.

Алгоритм проверки доступности учитывает услугу, длительность занятия, дату, рабочие часы, закрытые дни, список участников, вес каждого участника, допустимые услуги для лошади, статус лошади, уже существующие заявки и перерыв после предыдущей тренировки. Схема алгоритма приведена на рисунке 3.

Выбор услуги	- >	Выбор даты	- >	Формирование слотов
Слот	- >	Проверка рабочих часов и закрытых дат	- >	Проверка правил
Участники	- >	Подбор подходящих лошадей по весу и услугам	- >	Проверка занятости
Заявки	- >	Учет перерыва после занятия	- >	Доступен / недоступен с причиной

Рисунок 3 - Алгоритм проверки доступности временного слота

Если участников несколько, алгоритм подбирает лошадь для каждого участника. Одна и та же лошадь не назначается двум участникам одновременно. Если хотя бы для одного участника не найден подходящий ресурс, слот получает статус недоступного и отображает понятную причину: отсутствие свободных лошадей,

превышение допустимого веса, закрытый день, недоступность услуги или необходимость отдыха лошади.

Таблица 8 - Поддерживаемые правила бронирования в mock-логике

Правило	Как учитывается в интерфейсе
Рабочие дни и часы	Слоты создаются только внутри заданного диапазона времени
Закрытая дата	День исключается из записи или все слоты получают причину недоступности
Перерыв после занятия	Лошадь не считается доступной, если предыдущая тренировка закончилась менее чем за заданное число минут
Максимальный вес наездника	Участнику подбираются только лошади с достаточным maxRiderWeightKg
Разрешенные услуги для лошади	Лошадь участвует в подборе только для допустимых услуг
Статус лошади	Недоступные, лечащиеся или отдыхающие лошади исключаются из подбора
Конфликт существующей заявки	Подтвержденные и ожидающие заявки блокируют соответствующий интервал

3 РЕАЛИЗАЦИЯ WEB-ПРИЛОЖЕНИЯ

3.1 Структура проекта

Проект реализован как Vite-приложение с использованием React и TypeScript. Структура каталогов отражает разделение ответственности между страницами, компонентами, данными, сервисами, типами и стилями. Такой подход не усложняет учебный проект, но сохраняет возможность дальнейшего развития.

Таблица 9 - Структура каталога src

Каталог	Содержимое
app	Корневой компонент приложения и маршрутизация
pages	Страницы публичной части и административного раздела
components	Переиспользуемые компоненты интерфейса

components/booking	Форма записи и связанные элементы
components/admin	Панель admin-mode, inline-редактирование, загрузка изображений
data	Mock-данные услуг, отзывов, галереи, контактов, лошадей, заявок и правил
services	mock API layer, проверка доступности, хранение редактируемого контента
types	TypeScript-интерфейсы и union-типы
styles	Глобальные стили, адаптивная верстка и UI-состояния

3.2 Реализация публичных страниц

Главная страница знакомит пользователя с предприятием, показывает основные направления услуг, блок доверия, отзывы и быстрый переход к записи. Структура главной страницы соответствует пользовательскому сценарию: сначала клиент понимает назначение сайта, затем видит услуги и может перейти к оформлению заявки. Внешний вид главной страницы приведен на рисунке 4.

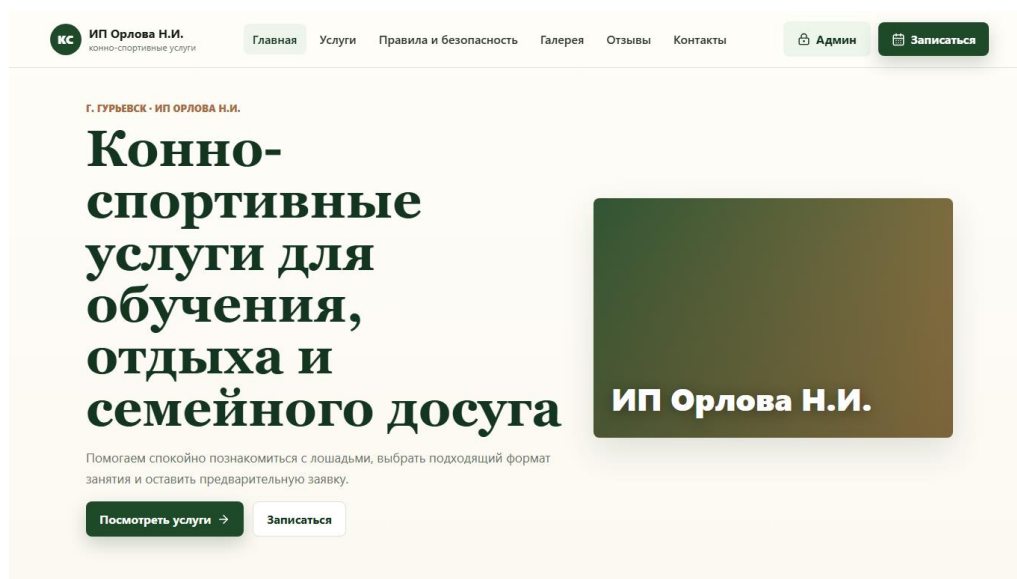


Рисунок 4 - Главная страница web-приложения

Каталог услуг отображает карточки с названием, описанием, длительностью, ценой, ограничениями и кнопками перехода к подробной карточке или записи. Фотографии услуг сделаны крупнее и имеют более горизонтальный формат, чтобы карточки лучше смотрелись на desktop и mobile. Фрагмент каталога представлен на рисунке 5.

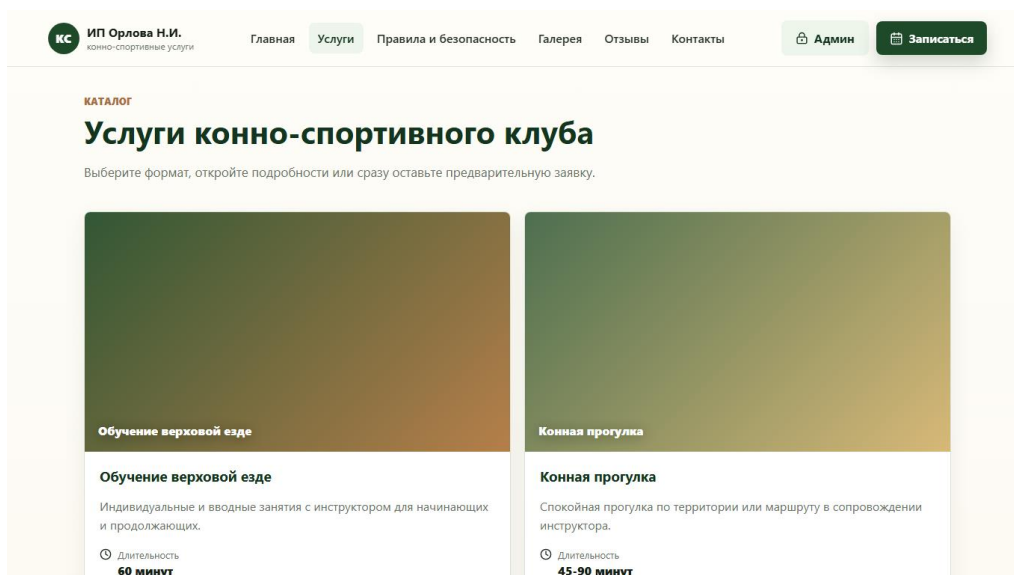


Рисунок 5 - Каталог услуг

Страница услуги содержит подробное описание, условия участия, подготовку, возрастные ограничения, правила безопасности и кнопку записи с передачей выбранной услуги. Это снижает неопределенность клиента до отправки заявки. Пример карточки услуги показан на рисунке 6.

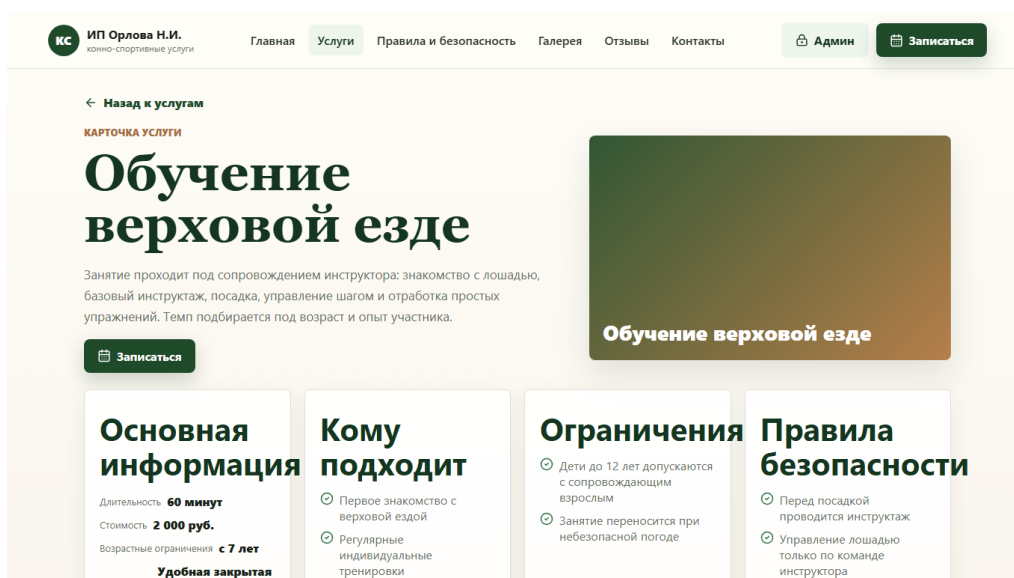


Рисунок 6 - Подробная карточка услуги

Раздел правил и безопасности объединяет требования к одежде, рекомендации перед занятием, возрастные ограничения, правила поведения рядом с лошадьми и FAQ. Такой раздел особенно важен для новых клиентов и родителей несовершеннолетних участников.

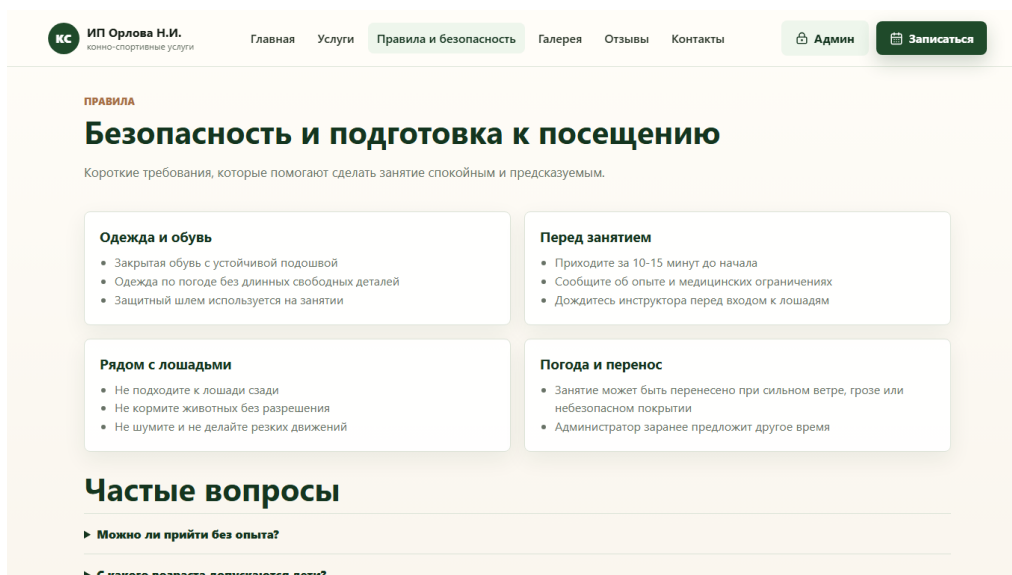


Рисунок 7 - Раздел правил и безопасности

Галерея оформлена не как простой набор папок, а как страница с тематическими заголовками и несколькими фотографиями в каждом разделе. Ниже сохраняются категории-папки, что удобно для дальнейшего наполнения через административный режим. Вид галереи приведен на рисунке 8.

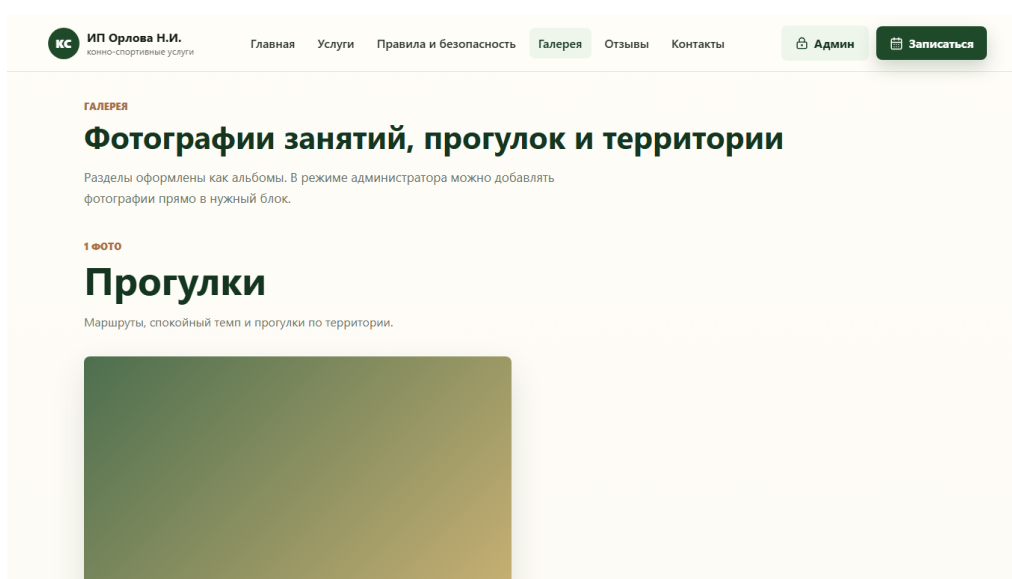


Рисунок 8 - Галерея с тематическими разделами

Страница контактов содержит адрес, телефон, режим обработки заявок, ссылки на связь и placeholder карты. На мобильных устройствах предусмотрены быстрые действия: позвонить, написать и построить маршрут. Вид страницы контактов показан на рисунке 9.

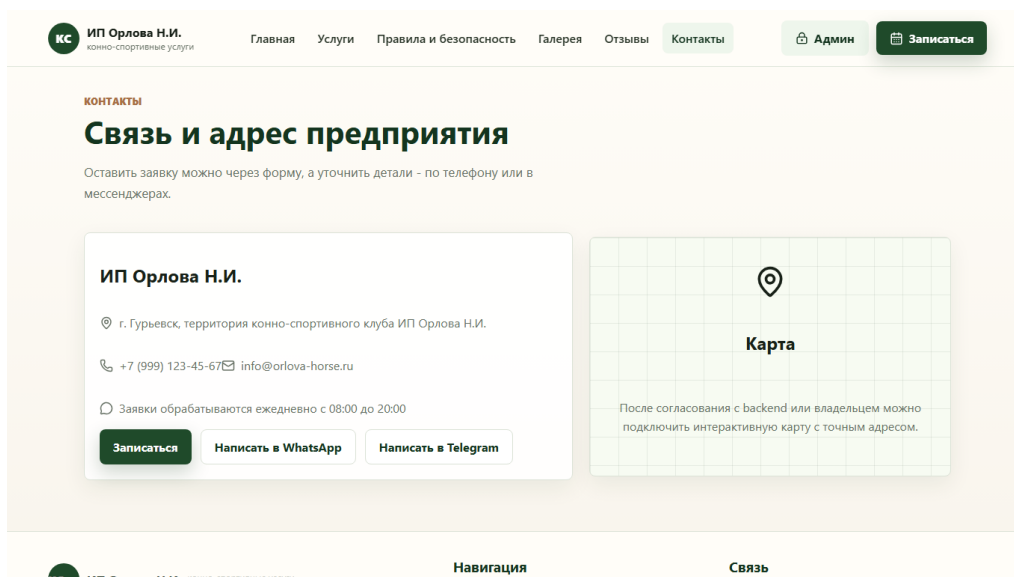


Рисунок 9 - Страница контактов

3.3 Реализация формы предварительной записи

Форма записи реализует не окончательное автоматическое бронирование, а отправку заявки на подтверждение администратором. Это важно с точки зрения корректного ожидания клиента: интерфейс сообщает, что заявка ожидает подтверждения, а не гарантирует посещение без проверки ресурсов.

В форме клиент выбирает услугу, дату, доступный временной слот, указывает контактные данные и добавляет одного или нескольких участников. Для каждого участника вводятся имя, возраст, вес, уровень подготовки и комментарий. При изменении состава участников список доступных слотов пересчитывается. Фрагмент формы показан на рисунке 10.

КС ИП Орлова Н.И.
конно-спортивные услуги

Главная Услуги Правила и безопасность Галерея Отзывы Контакты

Админ Записаться

ЗАПИСЬ

Предварительная заявка на занятие

Заполните форму, и администратор свяжется с вами для подтверждения времени.

Календарное бронирование

Выберите дату и слот. Система проверит лошадей, ограничения по весу, возрасту и перерывы после занятий.

Услуга *

Обучение верховой езде

Дата занятия

26 мая доступно	27 мая доступно	28 мая доступно	29 мая доступно
30 мая доступно	31 мая Выбранный день закрыт для записи	01 июн. доступно	02 июн. доступно

Выбранная услуга

Обучение верховой езде

Длительность 60 минут

Стоимость 2 000 руб.

Возраст с 7 лет

Как проверяется слот

1. Учитывается длительность услуги и рабочие часы.
2. Для каждого участника ищется подходящая свободная лошадь.
3. Проверяется вес, возраст, статус лошади и отдых

Рисунок 10 - Календарная форма предварительной записи

Клиентская валидация проверяет обязательные поля, формат телефона, наличие согласия на обработку персональных данных, корректность данных участников и доступность выбранного слота. При ошибке введенные данные не очищаются, а пользователю показывается понятное сообщение без технических деталей.

3.4 Реализация административного режима

В соответствии с дополнительными требованиями административная часть реализована не только как отдельная страница, но и как режим редактирования. После демонстрационного входа администратор получает плавающую панель, может перейти на публичную страницу и редактировать содержимое ближе к месту его отображения. Такой режим удобнее для изменения фото, текста и визуальных параметров главной страницы.

Демонстрационный вход не является полноценной авторизацией. В реальной системе проверка прав доступа должна выполняться backend-сервером, а соединение должно использовать HTTPS. В рамках frontend-проекта вход нужен для демонстрации сценария управления контентом. Экран административного режима представлен на рисунке 11.

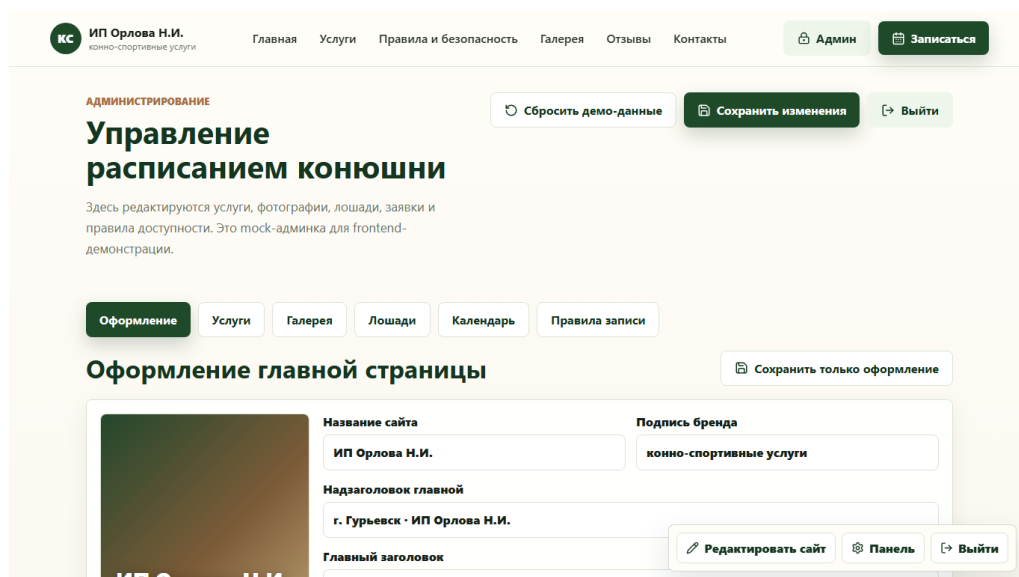


Рисунок 11 - Административный режим редактирования оформления

Админ-панель содержит вкладки «Оформление», «Услуги», «Галерея», «Лошади», «Календарь» и «Правила записи». На вкладке услуг можно менять названия, описания, цены, длительность, ограничения и фотографии через кнопку добавления файла. На вкладке галереи администратор может добавлять фотографии в тематические разделы и папки. На вкладке лошадей задаются статус, максимальный вес наездника, допустимые услуги и заметки.

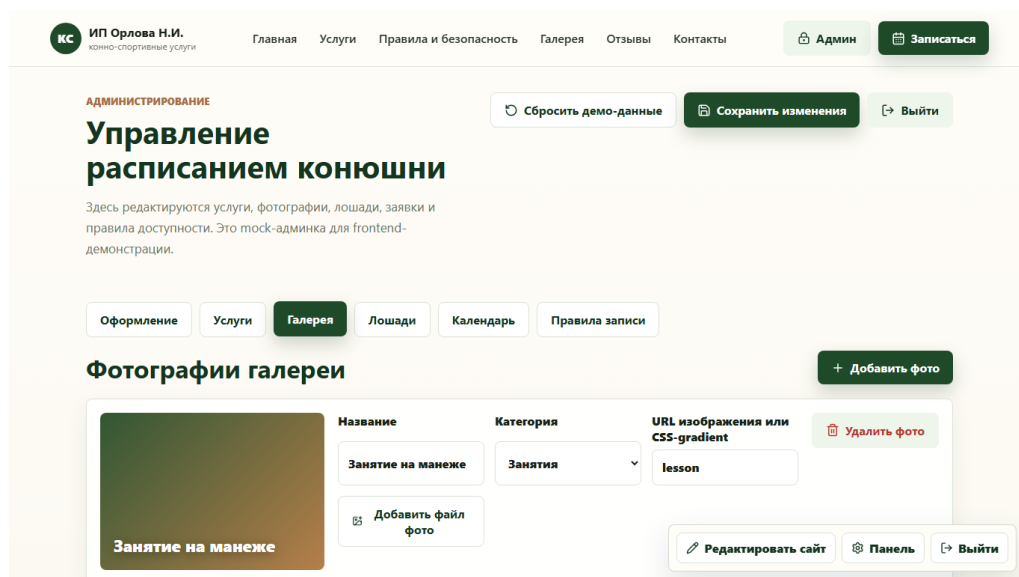


Рисунок 12 - Администрирование галереи

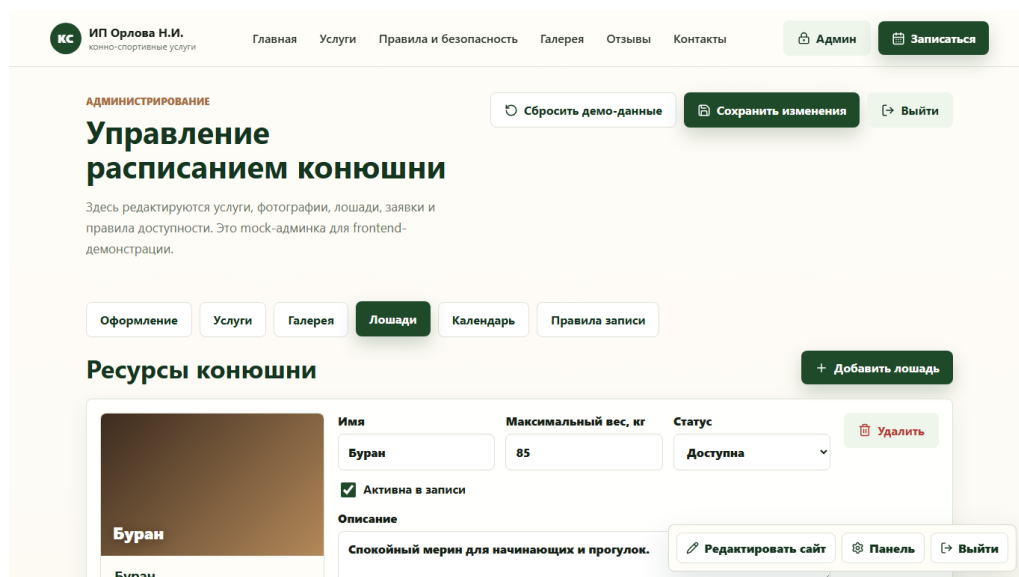


Рисунок 13 - Администрирование лошадей

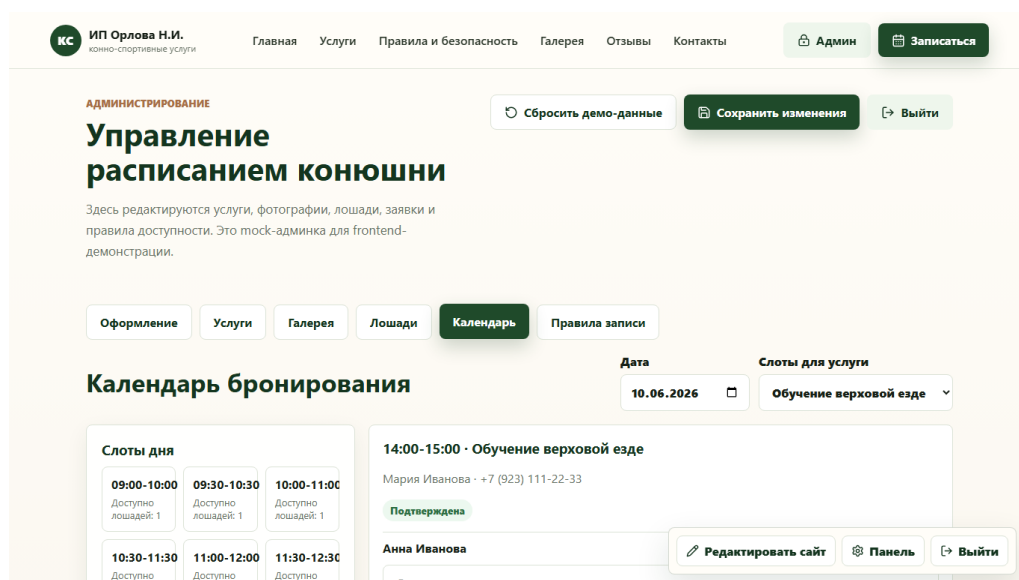


Рисунок 14 - Календарь заявок администратора

3.5 Адаптивность и пользовательский интерфейс

Верстка выполнена по принципу mobile-first. Карточки, формы, сетки галереи и административные блоки используют адаптивные CSS-сетки и ограничения ширины. На мобильных устройствах кнопки и поля формы имеют достаточный размер, навигация упрощается, а важный текст не размещается исключительно внутри изображений.

Визуальный стиль выбран спокойным и природным: используются глубокий зеленый цвет, теплый акцентный оттенок, светлый фон, крупные фотографии и ясные

СТА-кнопки. Такой стиль соответствует тематике конно-спортивных услуг и поддерживает ощущение безопасности и доверия.

3.6 Проверка работоспособности

Проверка проекта включала сборку приложения, просмотр основных страниц, отправку формы в успешном и ошибочном сценариях, проверку отображения причин недоступности слотов, а также проверку административных вкладок. Результаты основных проверок представлены в таблице 10.

Таблица 10 - Функциональные проверки

Проверка	Ожидаемый результат	Результат
Открытие главной страницы	Показывается основной блок, услуги, доверие и отзывы	Выполнено
Переход в каталог услуг	Отображаются карточки услуг с фото, ценами и кнопками	Выполнено
Открытие карточки услуги	Показываются подробности, ограничения и кнопка записи	Выполнено
Выбор услуги и даты в записи	Появляются доступные и недоступные слоты	Выполнено
Добавление нескольких участников	Для каждого участника доступны отдельные поля	Выполнено
Проверка веса участника	При отсутствии подходящей лошади показывается причина	Выполнено на тест-данных
Отправка заявки без согласия	Форма показывает ошибку и не отправляет данные	Выполнено
Успешная отправка заявки	Показывается сообщение об ожидании подтверждения администратора	Выполнено
Вход в админ-режим	Открываются вкладки управления и панель редактирования	Выполнено
Сборка проекта	Команда <code>npm run build</code> завершается без ошибок	Выполнено

3.7 Подробное техническое задание на клиентскую часть

Практическая реализация начиналась с уточнения технического задания на клиентскую часть. Поскольку backend разрабатывается отдельно, frontend должен не зависеть от конкретной серверной реализации, но обязан заранее определить структуру данных и список операций обмена. Поэтому техническое задание включает не только перечень страниц, но и требования к API-контракту, mock-логике, моделям TypeScript и обработке состояний.

Функциональные требования к публичной части включают отображение главной страницы, каталога услуг, подробной карточки услуги, страницы правил, галереи, отзывов, контактов и формы записи. Каждая страница должна иметь самостоятельный маршрут, чтобы пользователь мог открыть ее напрямую или поделиться ссылкой. Все пользовательские тексты должны быть на русском языке, а ключевые действия должны называться понятными словами: «Записаться», «Подробнее», «Посмотреть услуги», «Задать вопрос».

Требования к форме записи были расширены по сравнению с первоначальным вариантом. Вместо поля количества участников реализован список участников, где каждый участник имеет собственные параметры. Это позволяет родителю записать ребенка, организатору - группу людей, а администратору - получить структурированные сведения для проверки безопасности и подготовки занятия.

Требования к календарю включают выбор даты, отображение временных слотов, различение доступных и недоступных слотов, показ причин недоступности и повторную проверку перед отправкой заявки. Пользователь не должен получать впечатление, что запись подтверждена автоматически. Поэтому после отправки показывается статус «Заявка отправлена и ожидает подтверждения администратора».

Административные требования включают два режима работы: отдельную страницу управления и inline-режим редактирования публичных страниц. Отдельная страница удобна для списков, таблиц и правил, а inline-режим позволяет администратору видеть контекст изменения: заголовок, фотографию, текст или блок страницы редактируются ближе к месту отображения. Такое сочетание делает

админку более похожей на бизнес-функцию, а не на изолированный технический раздел.

Нефункциональные требования включают адаптивность, отсутствие критических ошибок в консоли, читаемую структуру проекта, типизацию данных, отсутствие хранения персональных данных клиента в localStorage, готовность к HTTPS API, обработку loading/error/success-состояний и возможность демонстрации проекта без backend. Эти требования проверяются при реализации и сборке приложения.

3.8 Проектирование пользовательских сценариев

Пользовательский сценарий нового клиента начинается с главной страницы. На этом этапе интерфейс должен быстро ответить на вопросы: какие услуги оказывает предприятие, где оно находится, безопасны ли занятия, можно ли прийти с ребенком и как оставить заявку. Поэтому на главной странице размещены основной визуальный блок, популярные услуги, преимущества, блок доверия, отзывы и переход к контактам.

Сценарий выбора услуги строится через каталог. Клиент просматривает карточки, сравнивает длительность, стоимость и ограничения. Если описание недостаточно, он открывает подробную карточку. На подробной карточке пользователь получает расширенную информацию: кому подходит услуга, что нужно подготовить, какие есть ограничения и правила безопасности. После этого кнопка записи переводит его в форму с уже выбранной услугой.

Сценарий предварительной записи начинается с выбора услуги и даты. После этого клиент видит слоты. Доступность слотов зависит от mock-логики, а не от статичного списка. Если клиент добавляет второго участника с большим весом, доступность может измениться. Это демонстрирует бизнес-логику конюшни: чем больше участников и чем жестче ограничения, тем меньше подходящих временных интервалов.

Сценарий администратора отличается от клиентского тем, что он направлен на поддержание актуальности данных. Администратор может изменить фото главной страницы, тексты страниц, цену услуги, описание, доступность, фотографии галереи, параметры лошадей и правила записи. В текущей версии изменения являются демонстрационными, но архитектурно они отделены от компонентов и могут быть заменены серверным хранением.

Особый сценарий связан с обработкой заявок. Администратор открывает календарь, видит заявки с разными статусами, может изменить статус, добавить комментарий и назначить лошадь участнику. Это важно для демонстрации того, что приложение не ограничивается отправкой формы, а учитывает дальнейшую работу предприятия после получения заявки.

3.9 Реализация слоя данных и mock API

Mock-данные вынесены в файл `src/data/mockData.ts`. В этом файле размещены услуги, отзывы, элементы галереи, контактная информация, правила безопасности, лошади, существующие заявки и правила бронирования. Компоненты не импортируют эти массивы напрямую. Это решение позволяет централизованно менять тестовые данные и избегать ситуации, когда бизнес-данные оказываются распределены по разным React-компонентам.

Сервис `src/services/mockApi.ts` имитирует работу backend API. Каждая функция возвращает `Promise` с объектом `ApiResponse`. Искусственная задержка позволяет компонентам использовать те же состояния загрузки, которые потребуются при настоящих HTTP-запросах. Например, `getServices` возвращает список услуг, `getServiceById` получает услугу по идентификатору, `createBookingRequest` проверяет доступность и создает mock-заявку.

Такой слой данных удобен для командной разработки. Backend-разработчик видит список функций и ожидаемые endpoints, а frontend-разработчик может продолжать работу над интерфейсом без ожидания готового сервера. После реализации backend внутренняя часть функций `mockApi` может быть заменена на

Fetch API, а внешние сигнатуры останутся прежними. Это снижает стоимость интеграции и уменьшает риск регрессий в компонентах.

Для административного режима используется `src/services/adminContent.ts`. Этот сервис хранит редактируемую версию контента в `localStorage`, но только для демонстрационных данных сайта: текста, цветов, изображений, услуг, лошадей, правил и заявок. Персональные данные клиента не должны сохраняться таким способом в рабочей версии. При подключении backend этот сервис также заменяется запросами к защищенным административным endpoint.

Слой данных дополнительно выполняет роль границы ответственности. React-компонент не должен знать, как именно создается заявка или откуда берутся правила записи. Компонент передает данные и получает результат. Благодаря этому форма записи остается компонентом интерфейса, а проверка доступности и создание заявки остаются задачами сервисного слоя.

3.10 Реализация алгоритма доступности

Алгоритм доступности реализован в `src/services/availabilityService.ts`. Его задача - рассчитать доступные даты, временные слоты, причины недоступности и предварительное назначение лошадей. На этапе frontend-разработки алгоритм работает с mock-данными, но структура функций соответствует будущему API: `getAvailableDates`, `getAvailableTimeSlots`, `checkBookingAvailability`, `findAvailableHorsesForParticipants`, `validateBookingRules` и `detectBookingConflicts`.

При формировании временных слотов сервис получает выбранную услугу и ее длительность. Затем учитываются рабочие часы и шаг слота. Например, если рабочий день начинается в 09:00, заканчивается в 19:00, услуга длится 60 минут, а шаг равен 30 минутам, то сервис создает последовательность возможных интервалов: 09:00-10:00, 09:30-10:30, 10:00-11:00 и так далее. Каждый интервал проходит проверку правил.

Правило закрытой даты проверяется до подбора лошадей. Если дата закрыта полностью, все слоты получают причину недоступности. Это полезно для выходных,

санитарных дней, плохой погоды или мероприятий, когда обычная запись невозможна. В админке такие исключения представлены через правила записи и могут быть расширены в будущей версии.

Подбор лошади выполняется для каждого участника отдельно. Сначала исключаются неактивные лошади и лошади со статусом `unavailable`, `treatment`, `rest` или `busy`. Затем проверяется, разрешена ли выбранная услуга для конкретной лошади. После этого вес участника сравнивается с `maxRiderWeightKg`. Если лошадь не проходит хотя бы одно условие, она не может быть назначена этому участнику.

Следующий этап - проверка пересечения с существующими заявками. Для лошади выбираются заявки на ту же дату со статусами `pending`, `confirmed` или `needs_clarification`. Если интервалы пересекаются, лошадь недоступна. Дополнительно учитывается перерыв после занятия: интервал отдыха расширяет время занятости лошади после завершения предыдущей тренировки.

Если для всех участников удалось подобрать разные подходящие лошади, слот считается доступным. Если хотя бы один участник не получил подходящий ресурс, слот помечается как недоступный. Причины собираются в массив строк, чтобы интерфейс мог показать пользователю конкретное объяснение. Такой формат легко передать через `backend API`: сервер может вернуть `isAvailable`, `reasons` и `suggestedAssignments`.

Алгоритм не претендует на полную промышленную оптимизацию, но демонстрирует основные ограничения предметной области. В дальнейшем `backend` может заменить жадный подбор на более сложный алгоритм распределения ресурсов, учитывать инструкторов, манежи, погоду, предпочтения клиентов и индивидуальные особенности лошадей. Клиентская часть при этом уже готова отображать результат проверки.

3.11 Реализация формы записи и валидации

Компонент `BookingForm` отвечает за основной пользовательский сценарий заявки. Он хранит состояние выбранной услуги, даты, слота, контактных данных

клиента, списка участников, комментария и согласия на обработку персональных данных. Для каждого участника создается уникальный идентификатор, чтобы поля корректно обновлялись и удалялись независимо от порядка в списке.

Добавление участника реализовано через кнопку «Добавить участника». Новый участник получает пустые поля и значение опыта по умолчанию. Удаление участника доступно для всех дополнительных участников, при этом в форме должен оставаться как минимум один участник. Такое ограничение предотвращает отправку заявки без фактических данных о наезднике.

Валидация выполняется до отправки. Проверяются имя клиента, телефон, выбранная услуга, дата, слот, согласие на обработку персональных данных, а также обязательные поля каждого участника. Возраст и вес должны быть положительными числами. Если ошибка относится к конкретному участнику, текст сообщения должен быть понятным: например, указать, что необходимо заполнить имя или вес участника.

Телефон проверяется на клиентской стороне по базовому правилу: значение должно содержать достаточное количество цифр. Такая проверка не заменяет серверную нормализацию номера, но помогает избежать очевидных ошибок ввода. В рабочей версии backend может привести телефон к единому формату, проверить длину и сохранить номер в базе данных.

После успешной проверки форма вызывает `createBookingRequest`. Mock API дополнительно выполняет `checkBookingAvailability`. Это важно, потому что доступность могла измениться после выбора слота. Например, другой клиент мог занять слот или администратор мог закрыть день. В учебной версии это имитируется локально, а в рабочей версии такую проверку должен выполнять сервер непосредственно перед созданием заявки.

При успешной отправке пользователь видит сообщение о том, что заявка отправлена и ожидает подтверждения администратора. Такая формулировка соответствует бизнес-процессу предприятия: окончательное подтверждение зависит от администратора, состояния лошадей и расписания. При ошибке данные формы не

очищаются, чтобы клиент мог исправить проблему и повторить попытку без повторного ввода всей информации.

3.12 Реализация административного редактирования контента

Административный режим был доработан с учетом требования редактировать сайт не только через отдельные вкладки, но и непосредственно на публичных страницах. После входа появляется плавающая панель, через которую администратор может включить режим редактирования, перейти в панель управления или выйти. Такой подход делает управление контентом ближе к визуальному редактору.

Редактирование текстов страниц реализовано через отдельные компоненты и данные SiteContent. Для каждой публичной страницы предусмотрены редактируемые eyebrow, title и text. Это позволяет изменять смысловые заголовки без изменения исходного кода. В дальнейшем эти значения должны храниться на сервере и редактироваться через защищенный административный API.

Редактирование изображений поддерживает не только ввод пути, но и добавление файла. Компонент ImageUploadButton считывает выбранный файл и преобразует его во временный data URL. Для демонстрации этого достаточно: администратор видит, как фото меняется в интерфейсе. В рабочей версии вместо data URL должен использоваться endpoint загрузки файла, который вернет постоянный URL изображения.

Вкладка «Оформление» управляет названием сайта, подписью бренда, текстами главной страницы, фоновым изображением или градиентом, позицией изображения и основными цветами. Это позволяет показать, что администратор может менять не только контент каталога, но и визуальную подачу главной страницы. При этом дизайн остается в рамках общей стилистики проекта.

Вкладка «Услуги» позволяет изменять название, краткое и полное описание, длительность, цену, возрастные ограничения, подготовку, ограничения, доступность и фото услуги. Это важная бизнес-функция, потому что цены и условия услуг могут

меняться чаще, чем структура сайта. Возможность редактирования таких данных повышает практическую ценность приложения.

Вкладка «Галерея» была переосмыслена как управляемый раздел с тематическими блоками. На публичной странице отображаются заголовки, например «Прогулки», «Занятия», «Фотосессии», и несколько фотографий внутри каждого раздела. Внизу сохраняются категории-папки, где фотографии могут быть сгруппированы по типу. Для администратора важно иметь возможность добавлять файлы в эти разделы без изменения кода.

Вкладка «Лошади» отражает специфику конюшни. В ней администратор может добавить лошадь, изменить имя, описание, фотографию, максимальный вес наездника, допустимые услуги, статус, активность и заметку. Эти данные влияют на алгоритм доступности, поэтому управление лошадьми является не декоративной, а ключевой функциональной частью проекта.

Вкладка «Календарь» предназначена для просмотра заявок, изменения статуса, комментария администратора и назначения лошадей. В учебной версии календарь работает на mock-данных, но показывает будущий процесс обработки заявки. Администратор может увидеть, какие участники записаны, какие лошади назначены и требуется ли уточнение.

Вкладка «Правила записи» позволяет редактировать правила, влияющие на доступность. На первом этапе правила представлены в упрощенном виде: рабочие часы, закрытые даты, перерыв между тренировками, недоступность лошади и другие исключения. В дальнейшем этот раздел может стать полноценным интерфейсом расписания и ресурсного планирования.

3.13 Реализация публичных разделов и визуальной структуры

Главная страница построена как первый экран с сильным визуальным сигналом о тематике конюшни. В hero-блоке используются название предприятия, краткое описание и две основные кнопки. Ниже расположены популярные услуги, блок

преимуществ, правила доверия и отзывы. Такая структура соответствует макетам из практики, но визуально приведена к более аккуратному адаптивному виду.

Каталог услуг реализован через список карточек ServiceCard. Каждая карточка получает данные из Service и не содержит собственных mock-значений. Фотографии услуг сделаны крупнее и менее вертикальными, поскольку прежний формат выглядел сжатым и плохо использовал пространство карточки. Изображения получают устойчивые пропорции, чтобы hover-состояния и подписи не меняли размеры блока.

Страница услуги использует идентификатор из маршрута. Если услуга не найдена, пользователь должен получить понятное состояние ошибки. Если данные загружаются, показывается loading-состояние. Это важно для будущей интеграции, потому что запрос к серверу может завершиться ошибкой или вернуть пустой результат.

Раздел правил и безопасности построен из отдельных секций и FAQ. Такая структура удобнее длинного текста, потому что пользователь может быстро найти требования к одежде, поведению рядом с лошадьми, возрастным ограничениям и переносу занятия из-за погоды. Для конюшни этот раздел выполняет не только информационную, но и профилактическую функцию.

Галерея оформлена как визуальный раздел, а не как техническое хранилище изображений. Крупные тематические блоки помогают показать атмосферу занятий, прогулок и фотосессий. Категории внизу страницы сохраняют идею папок и могут использоваться для будущей фильтрации или загрузки фотографий по разделам.

Страница отзывов использует карточки ReviewCard с именем клиента, датой, текстом и оценкой. Отзывы повышают доверие к предприятию, особенно если пользователь впервые выбирает занятие с лошадьми. В рабочей версии отзывы могут модерироваться администратором, но в рамках текущей frontend-части используются mock-данные.

Страница контактов объединяет адрес, телефон, режим обработки заявок, ссылки на мессенджеры и placeholder карты. На мобильной версии важны быстрые

действия: звонок, сообщение и маршрут. Это снижает путь пользователя, который уже принял решение связаться с предприятием.

3.14 Проверка адаптивности и качества интерфейса

Проверка адаптивности проводилась на уровне структуры CSS и визуального просмотра основных страниц. Сетки услуг и галереи перестраиваются в зависимости от ширины экрана. Формы занимают доступную ширину, а поля и кнопки остаются достаточно крупными для мобильного взаимодействия. Текстовые блоки имеют ограничения ширины, чтобы строки не становились слишком длинными на desktop.

Особое внимание уделено форме записи, поскольку она содержит больше всего интерактивных элементов. На мобильном устройстве список участников должен оставаться читаемым, кнопка добавления участника должна быть заметной, а сообщения об ошибках должны находиться рядом с действием пользователя. Недоступные слоты отображаются так, чтобы клиент видел причину, но не путал их с доступными кнопками.

Административные формы также проверяются на устойчивость к большому количеству полей. Вкладки помогают разделить управление на смысловые блоки. Если все настройки разместить на одной странице, интерфейс стал бы перегруженным. Поэтому вкладочная структура сохранена, но дополнена inline-режимом редактирования публичных страниц.

Качество кода проверяется через сборку TypeScript и Vite. Команда `npm run build` выполняет проверку TypeScript-проекта и производственную сборку. Успешное выполнение сборки показывает, что в коде нет критических типовых ошибок и приложение может быть подготовлено к размещению. Для дипломного проекта это является минимально необходимой технической проверкой.

Функциональное тестирование выполнялось вручную по основным сценариям. Проверялись переходы по маршрутам, загрузка данных, открытие карточек услуг, выбор даты и слота, добавление участников, ошибки валидации, успешная отправка заявки, вход в административный режим, изменение вкладок и отображение данных

лошадей. Результаты таких проверок отражены в таблице функциональных проверок основной части.

Ограничением текущего тестирования является отсутствие реального backend. Поэтому проверка сетевых ошибок и конфликтов выполняется через mock-логику. После интеграции с сервером необходимо добавить тесты для реальных HTTP-ответов, ошибок авторизации, загрузки файлов, одновременной записи нескольких клиентов и серверного отклонения заявки из-за конфликта расписания.

3.15 Подготовка проекта к backend-интеграции

Подготовка к backend-интеграции заключается в том, что frontend уже содержит список функций, соответствующих будущим endpoint. Это позволяет backend-разработчику реализовывать сервер по заранее понятному контракту. Для каждого endpoint нужно согласовать метод, URL, формат запроса, формат ответа, коды ошибок и правила отображения сообщений пользователю.

Для услуг необходимо хранить идентификатор, название, краткое и полное описание, длительность в минутах, цену, изображение, возрастные ограничения, подготовку, ограничения, список подходящих пользователей, правила безопасности и доступность. Backend должен отдавать эти данные в формате, совместимом с Service, чтобы каталог и карточка услуги не требовали переписывания.

Для заявок backend должен принимать BookingRequest и создавать Booking со статусом pending. Сервер должен повторно проверить услугу, дату, слот, участников, вес, правила и доступные ресурсы. Если заявка не может быть создана, сервер возвращает понятную причину, которую frontend покажет пользователю. Если заявка создана, клиент видит сообщение об ожидании подтверждения.

Для загрузки изображений требуется отдельный endpoint. Frontend должен отправлять файл, а сервер - проверять тип, размер, права пользователя и сохранять файл в постоянном хранилище. Ответом должен быть URL изображения, который затем сохраняется в услуге, галерее, лошади или настройках главной страницы. Такой подход лучше, чем хранение больших data URL в состоянии приложения.

Для административного режима необходима защищенная авторизация. В учебной версии используется демонстрационный пароль, но в рабочей версии должны быть реализованы учетные записи, роли, токены или cookie-сессии, защита от несанкционированного доступа и проверка прав на каждом административном endpoint. Также желательно фиксировать, кто и когда изменил цену, фото, правило или статус заявки.

Для календаря и доступности backend должен стать единственным источником истины. Frontend может предварительно показывать слоты, но окончательное решение о доступности принимает сервер. Это особенно важно при одновременной работе нескольких клиентов: два пользователя могут выбрать один слот, и только серверная транзакция способна гарантировать отсутствие двойного бронирования.

3.16 Ограничения текущей реализации и направления развития

Текущая реализация является клиентской частью и демонстрационным прототипом. Она не содержит базы данных, настоящей авторизации, серверного хранения файлов, уведомлений и промышленной обработки персональных данных. Эти ограничения не являются ошибкой проекта, поскольку они соответствуют распределению работ между участниками команды и границам выпускной квалификационной работы.

Mock-логика доступности показывает основные правила, но не учитывает всех возможных факторов реальной конюшни. В будущем можно добавить учет инструкторов, площадок, погодных условий, индивидуальных медицинских ограничений, предпочтений клиента, повторных занятий, абонементов и оплаты. Для этого потребуется расширить модели данных и серверный алгоритм.

Административный режим сейчас предназначен для демонстрации управления контентом. После подключения backend он должен быть разделен на роли: администратор, инструктор, возможно владелец предприятия. Администратор сможет менять контент и заявки, инструктор - видеть расписание и назначенных участников, владелец - анализировать спрос и эффективность услуг.

Возможным направлением развития является личный кабинет клиента. Однако он не был добавлен в текущую версию, чтобы не усложнять архитектуру сверх уровня дипломного frontend-проекта. На первом этапе достаточно предварительной заявки без регистрации, так как это снижает барьер входа и подходит для малого предприятия.

Еще одно направление развития - система уведомлений. После подтверждения заявки клиент может получить SMS, сообщение в мессенджере или письмо. При переносе из-за погоды уведомление должно объяснять причину и предлагать выбрать другой слот. Frontend уже содержит статусы заявок, поэтому в будущем эти статусы можно связать с уведомлениями.

Таким образом, практическая часть демонстрирует не только набор страниц, но и основу будущей информационной системы. Реализованы интерфейсные сценарии, mock API, модели данных, алгоритм доступности и административное управление. Это позволяет использовать проект как базу для последующей серверной разработки и внедрения на предприятии.

3.17 Информационное обеспечение разработанного приложения

Информационное обеспечение web-приложения представляет собой совокупность данных, которые используются для отображения страниц, формирования заявки, проверки доступности и администрирования. В проекте эти данные разделены на несколько групп: справочные данные, пользовательские данные, ресурсные данные, календарные данные и редактируемый контент сайта.

Справочные данные включают перечень услуг, правила посещения, FAQ, отзывы, контактную информацию и элементы галереи. Эти данные нужны для публичных страниц и не требуют ввода клиента. Пользовательские данные появляются в момент заполнения формы записи: имя клиента, телефон, параметры участников, комментариев и согласие на обработку персональных данных.

Ресурсные данные описывают лошадей. Для каждой лошади задаются имя, описание, изображение, максимальный вес наездника, допустимые услуги, статус,

активность и заметка администратора. Эти данные используются не только в админке, но и в алгоритме доступности. Поэтому изменение статуса или веса лошади сразу влияет на расчет слотов.

Календарные данные включают заявки и правила бронирования. Заявка содержит дату, время начала и окончания, услугу, клиента, участников, назначенных лошадей, статус и комментарии. Правила бронирования определяют рабочие часы, закрытые даты, перерывы и индивидуальные исключения. Вместе эти данные формируют основу будущего расписания.

Редактируемый контент сайта представлен моделью SiteContent. В нее входят название сайта, подпись бренда, тексты главной страницы, изображение hero-блока, позиция изображения, цвета и тексты заголовков публичных страниц. Такой подход позволяет отделить контент от кода и показать, как администратор сможет управлять страницами без изменения React-компонентов.

В текущей реализации источником данных являются mock-массивы и localStorage для демонстрационного администрирования. После подключения backend источником истины должна стать база данных. Frontend при этом продолжит использовать те же модели, но будет получать их через API. Это подтверждает правильность выбранного разделения между data, services и components.

3.18 Программное обеспечение и структура компонентов

Программное обеспечение клиентской части построено на Vite, React и TypeScript. Точка входа src/main.tsx подключает React-приложение к DOM. Компонент src/app/App.tsx содержит маршруты и определяет, какие страницы открываются по соответствующим адресам. Компонент Layout обеспечивает общую структуру с шапкой, навигацией, футером и административной панелью при необходимости.

Компоненты пользовательского интерфейса вынесены в отдельные файлы. Button задает единый вид кнопок, SectionTitle используется для заголовков разделов, FAQItem - для раскрывающихся вопросов, States - для состояний загрузки и ошибки.

Такой набор простых компонентов помогает сохранять единый стиль и уменьшает повторение разметки.

Компоненты предметной области включают `ServiceCard`, `BookingForm`, `GalleryGrid` и `ReviewCard`. Они получают данные через свойства и не содержат собственных источников данных. Например, `ServiceCard` отображает услугу, но не знает, откуда она получена. `BookingForm` управляет вводом клиента, но вызывает сервисы для проверки доступности и отправки.

Административные компоненты включают `AdminModeToolbar`, `EditablePageTitle` и `ImageUploadButton`. `AdminModeToolbar` отвечает за плавающую панель редактирования, `EditablePageTitle` - за inline-редактирование заголовков страниц, `ImageUploadButton` - за выбор изображения с устройства. Эти компоненты отделены от публичных компонентов, что упрощает развитие админ-функций.

Стили находятся в `src/styles/global.css`. Для дипломного проекта выбран обычный CSS, а не сложная система CSS-in-JS, чтобы структура оставалась понятной. При этом стили организованы по смысловым блокам: `layout`, `hero`, `cards`, `forms`, `gallery`, `admin`, `responsive media queries`. Такой подход достаточен для проекта и не добавляет лишней зависимости.

Сборка проекта выполняется командой `npm run build`. Она запускает TypeScript-проверку и `Vite build`. Наличие успешной сборки подтверждает, что приложение может быть подготовлено к публикации как статический frontend, а все ошибки типов должны быть устранены до передачи проекта на интеграцию.

3.19 Алгоритмическое обеспечение и обработка исключений

Алгоритмическое обеспечение в проекте сосредоточено вокруг проверки доступности. Основные функции `availabilityService` имеют отдельные зоны ответственности. `getAvailableDates` формирует список дат, которые можно показать клиенту. `getAvailableTimeSlots` строит интервалы на выбранную дату. `checkBookingAvailability` выполняет итоговую проверку заявки перед отправкой.

Функция `validateBookingRules` проверяет правила, не связанные напрямую с подбором лошадей: существование услуги, закрытые даты, рабочие дни, заполненность участников и соответствие базовым условиям. Если нарушено хотя бы одно правило, функция возвращает массив причин, а форма может показать эти причины пользователю.

Функция `findAvailableHorsesForParticipants` отвечает за ресурсный подбор. Она получает заявку, определяет начало и конец слота, затем для каждого участника ищет подходящую лошадь. При поиске исключаются уже использованные лошади, чтобы один ресурс не был назначен двум участникам одновременно.

Функция `detectBookingConflicts` предназначена для выявления конфликтов с существующими заявками. В `mock`-версии она использует локальные массивы, но в `backend`-версии должна работать с базой данных и транзакциями. Это важно, потому что проверка и создание заявки должны выполняться атомарно: между проверкой и записью другой клиент не должен успеть занять тот же ресурс.

Обработка исключений строится вокруг понятных причин. Если день закрыт, пользователь видит сообщение о закрытии дня. Если нет подходящей лошади по весу, причина указывает именно на весовое ограничение. Если лошадь занята или должна отдохнуть, причина объясняет недоступность интервала. Такой подход улучшает UX и снижает количество дополнительных вопросов.

Алгоритмическое обеспечение не смешано с `React`-компонентами. Это принципиально важно для качества проекта: компонент формы отвечает за отображение и ввод, а сервис доступности - за правила предметной области. Такое разделение облегчает тестирование и замену `mock`-логики на серверные запросы.

3.20 Организационное обеспечение и роли пользователей

Организационное обеспечение описывает, как разработанное приложение вписывается в работу предприятия. В проекте выделяются три основные роли: клиент, администратор и `backend`-разработчик как участник дальнейшей интеграции.

В будущем также могут появиться роли инструктора и владельца предприятия, но для текущей версии они не являются обязательными.

Клиент использует публичные страницы и форму записи. Его задача - получить информацию, выбрать услугу, понять правила, указать данные участников и отправить заявку. Клиент не управляет расписанием напрямую и не получает окончательное подтверждение автоматически. Это соответствует реальному процессу, где администратор должен проверить заявку.

Администратор использует режим редактирования и панель управления. Его задачи - поддерживать актуальность описаний, цен, фотографий, списка лошадей, правил и заявок. В демонстрационной версии администраторские изменения сохраняются локально, но бизнес-сценарий уже показывает, какие операции нужны предприятию.

Backend-разработчик получает от frontend-части готовый контракт: типы данных, ожидаемые endpoint, статусы, поля и сценарии ошибок. Это снижает неопределенность при командной разработке. Вместо устных договоренностей backend может ориентироваться на существующие mock-функции и заменить их реальными запросами.

Организационно важно, чтобы после внедрения администратор понимал разницу между заявкой и подтвержденной записью. Интерфейс поддерживает статусы pending, confirmed, rejected, needs_clarification и cancelled. Эти статусы позволяют отразить реальные этапы обработки: ожидание, подтверждение, отказ, уточнение и отмена.

3.21 Детализация тестовых сценариев

Для проверки пользовательского сценария был сформирован набор ручных тестов. Первый сценарий связан с новым клиентом: открыть главную страницу, перейти в каталог, открыть карточку услуги, вернуться к каталогу и перейти к записи. Ожидаемый результат - все переходы выполняются без ошибки, а выбранная услуга передается в форму.

Второй сценарий проверяет форму записи с одним участником. Пользователь выбирает услугу, дату, слот, вводит имя, телефон, данные участника, ставит согласие и отправляет заявку. Ожидаемый результат - появление сообщения об ожидании подтверждения администратора. Если убрать согласие или телефон, форма должна показать ошибку.

Третий сценарий проверяет нескольких участников. Пользователь добавляет второго участника, вводит разные возраст, вес и опыт. После изменения веса слоты пересчитываются. Если для одного участника нет подходящей лошади, выбранный слот не должен быть доступен или должна быть показана причина.

Четвертый сценарий проверяет административный режим. Пользователь входит с демонстрационным паролем, открывает вкладку оформления, изменяет текст, добавляет изображение, переходит на публичную страницу и видит измененный контент. Затем проверяются вкладки услуг, галереи, лошадей, календаря и правил записи.

Пятый сценарий проверяет устойчивость навигации. Пользователь напрямую открывает адрес /services, /rules, /gallery, /contacts и /admin. Каждая страница должна загружаться как отдельный маршрут. Это важно для SPA-приложения, потому что маршрутизация должна корректно работать не только при переходах из меню, но и при прямом открытии ссылки.

Шестой сценарий связан с ошибочными данными. Проверяется пустое имя клиента, короткий телефон, отсутствие участников, пустое имя участника, нулевой возраст, нулевой вес и отсутствие выбранного слота. В каждом случае интерфейс должен показать сообщение, а ранее введенные данные должны остаться в форме.

Седьмой сценарий связан с визуальной проверкой. Страницы открываются на desktop-ширине и мобильной ширине. Проверяется отсутствие наложения текста, читаемость карточек, доступность кнопок, корректность сеток и видимость основных действий. Такая проверка особенно важна для дипломной демонстрации, потому что комиссия оценивает не только код, но и готовность интерфейса к использованию.

3.22 Эксплуатационные требования к будущей версии

Для будущей эксплуатации необходимо предусмотреть размещение frontend-приложения на web-сервере или статическом хостинге. Поскольку Vite формирует статическую сборку, итоговые файлы могут быть размещены отдельно от backend. Backend API при этом должен быть доступен по защищенному адресу и корректно обрабатывать CORS или находиться под тем же доменом.

Необходимо предусмотреть резервное копирование данных backend: услуг, заявок, лошадей, правил, фотографий и административных настроек. Frontend не решает эту задачу напрямую, но его модели показывают, какие сущности должны быть защищены от потери. Особенно важны заявки и история изменений статусов.

Для изображений следует определить ограничения: допустимые форматы JPG, PNG или WebP, максимальный размер файла, рекомендуемые пропорции для услуг, лошадей, галереи и hero-блока. На frontend уже предусмотрена загрузка файла, но рабочая версия должна выполнять оптимизацию и хранить изображения на сервере или в объектном хранилище.

Для доступности необходимо продолжить работу над семантической разметкой, фокусом клавиатуры, контрастностью и текстовыми альтернативами изображений. В текущей версии эти требования учтены на базовом уровне, но при промышленном внедрении рекомендуется провести отдельную проверку по WCAG 2.2.

Для сопровождения проекта важно вести документацию API и фиксировать изменения контрактов. Если backend изменит поле `serviceId` или формат времени, frontend должен быть обновлен согласованно. Поэтому TypeScript-типы могут стать общей основой для документации и автоматической проверки совместимости.

В результате практическая часть получает заверченный вид: она описывает не только экраны, но и данные, алгоритмы, роли, тестирование, эксплуатацию и будущую интеграцию. Это соответствует методическому требованию раскрыть

специальную часть через разные виды обеспечения, а не ограничиваться набором скриншотов.

3.23 Подробное описание реализации API-контракта

API-контракт в проекте реализован как набор функций, каждая из которых соответствует будущему серверному endpoint. Такой способ удобен на этапе frontend-разработки, потому что компонент вызывает функцию, а не формирует адрес запроса самостоятельно. В результате логика работы с сетью сосредоточена в одном месте и может быть заменена без массового изменения компонентов.

Функции `getServices` и `getServiceById` используются каталогом и карточкой услуги. В будущем они должны выполнять GET-запросы к серверу. Сервер должен возвращать только доступные пользователю поля, а административные поля, например служебные заметки, не должны попадать в публичный ответ без необходимости.

Функция `createBookingRequest` является ключевой для формы записи. В mock-версии она вызывает проверку доступности и создает тестовую заявку. В backend-версии она должна отправлять POST-запрос, а сервер должен выполнить валидацию и создать запись со статусом `pending`. Если доступность изменилась, сервер возвращает ошибку бизнес-логики, а frontend отображает причину.

Функции `getHorses`, `createHorse`, `updateHorse` и `deleteHorse` относятся к административной части. В рабочей версии они должны быть защищены авторизацией. Удаление лошади желательно реализовывать не физическим удалением, а деактивацией, если лошадь уже участвовала в заявках. Это позволит сохранить историю расписания.

Функции `getBookings`, `getBookingById`, `updateBookingStatus` и `assignBookingHorses` описывают работу с заявками администратора. Они должны учитывать статусы и права доступа. Например, обычный клиент не должен видеть чужие заявки, а администратор должен видеть календарь и иметь возможность менять статус.

Функции `getAvailability` и `checkAvailability` разделяют предварительный расчет слотов и итоговую проверку заявки. Первую функцию удобно вызывать при выборе даты, вторую - перед отправкой. Такое разделение снижает количество лишних запросов и одновременно защищает от ситуации, когда пользователь отправляет устаревший слот.

Функции `getBookingRules`, `createBookingRule`, `updateBookingRule` и `deleteBookingRule` относятся к управлению расписанием. В текущей версии правила представлены упрощенно, но будущий backend может хранить их в отдельной таблице и применять на сервере. Frontend уже готов отображать список правил и передавать изменения.

3.24 Детализация реализации административных вкладок

Вкладка «Оформление» реализует редактирование главной страницы и базовых визуальных параметров. Администратор может изменить название, подпись, надзаголовок, главный заголовок, основной текст, фоновое изображение, позицию изображения и цвета. Изменения показываются рядом с `preview`, поэтому администратор сразу понимает, как будет выглядеть результат.

Вкладка «Услуги» представляет список редактируемых карточек. Для каждой услуги доступны поля описания и параметры записи. Кнопка добавления файла позволяет выбрать изображение с устройства, а не вводить путь вручную. Это было важным уточнением требований, поскольку реальный администратор не должен знать внутреннюю структуру файлов проекта.

Вкладка «Галерея» поддерживает загрузку фотографий в тематические разделы. Публичная страница галереи показывает эти разделы как красивые блоки с заголовками, а не только как папки. Папки категорий при этом остаются внизу страницы и могут использоваться для сортировки, хранения или будущей фильтрации изображений.

Вкладка «Лошади» содержит параметры, которые влияют на алгоритм доступности. Если администратор переводит лошадь в статус лечения или отключает

ее из записи, она перестает участвовать в подборе. Если меняется максимальный вес, это влияет на слоты для участников с большим весом. Тем самым вкладка показывает связь администрирования с бизнес-логикой.

Вкладка «Календарь» показывает заявки и позволяет менять их статус. Для заявки отображаются клиент, телефон, услуга, дата, время, участники и назначенные лошади. В будущей версии здесь может появиться недельное представление, drag-and-drop изменения времени и уведомления клиентов. В текущем проекте вкладка демонстрирует основу процесса обработки.

Вкладка «Правила записи» управляет ограничениями расписания. Администратор может видеть активные правила и менять их параметры. Это важно, потому что правила записи меняются чаще, чем код приложения: может измениться рабочий график, длительность отдыха, закрытые даты или недоступность конкретной лошади.

3.25 Подробное описание визуальной адаптации макетов

Исходные макеты из материалов практики использовались как основа структуры страниц. При реализации не ставилась задача pixel-perfect копирования, так как сайт должен быть адаптивным и аккуратно работать на разных устройствах. Поэтому сохранена логика блоков, но улучшены отступы, сетки, размеры изображений и визуальная иерархия.

Главная страница получила более выразительный первый экран, но без превращения в отдельный маркетинговый лендинг. Основные действия находятся вверху, а следующий контент частично виден ниже. Это помогает пользователю понять, что сайт является рабочим приложением с каталогом и записью, а не только рекламной страницей.

Карточки услуг были переработаны с учетом замечания о слишком вертикальных фотографиях. Изображения получили более широкие пропорции и стали крупнее. Это улучшило восприятие каталога, потому что пользователь быстрее

связывает услугу с визуальным примером и не воспринимает карточку как узкий фрагмент.

Галерея была доработана существенно. Вместо технического списка категорий она стала визуальным разделом с заголовками и группами фотографий. Это делает страницу полезной для клиента, который хочет увидеть атмосферу занятий, прогулок, фотосессий и территории. Административная загрузка фотографий делает этот раздел поддерживаемым.

Форма записи визуально разделена на этапы: выбор услуги и даты, выбор слота, данные клиента, участники и комментарий. Такое разделение снижает когнитивную нагрузку. Пользователь видит, что ему нужно заполнить, а при ошибке получает сообщение в контексте формы.

Админ-режим визуально отличается от публичного режима, но не ломает структуру сайта. Плавающая панель расположена так, чтобы не закрывать основной контент полностью. Поля редактирования имеют практичный вид, а кнопки загрузки файлов выделены как действия администратора.

3.26 Контроль соответствия требованиям ВКР

Итоговый проект соответствует требованию реализации только клиентской части. Backend, база данных и настоящая авторизация не создавались. Вместо них подготовлены mock API, TypeScript-модели, демонстрационный административный режим и структура данных для будущей интеграции. Это соответствует исходному распределению задач между участниками команды.

Проект соответствует требованию адаптивности: страницы построены на гибких сетках, кнопки и поля рассчитаны на мобильное взаимодействие, изображения имеют устойчивые пропорции. Важные тексты не размещаются только внутри картинок, а значит остаются доступными и редактируемыми.

Проект соответствует требованию обработки состояний: на страницах предусмотрены loading и error-состояния, а форма записи использует idle, loading,

success и error. Пользователь получает понятное сообщение при успешной отправке и при ошибке, а данные формы не очищаются без необходимости.

Проект соответствует требованию вынесения mock-данных: данные находятся в `src/data/mockData.ts` и не смешаны с компонентами. API layer находится в `src/services/mockApi.ts`, а бизнес-логика доступности - в `src/services/availabilityService.ts`. Это делает структуру понятной для демонстрации и дальнейшей разработки.

Проект соответствует требованию административного управления: реализован вход, вкладки управления и inline-режим. Администратор может менять фото, тексты, услуги, цены, галерею, лошадей, заявки и правила. В рабочей версии этот функционал должен быть подключен к защищенному backend.

Проект соответствует требованию учета специфики конюшни: форма поддерживает нескольких участников, вес и уровень подготовки, а алгоритм доступности учитывает лошадей, допустимые услуги, статусы, занятость и отдых. Это отличает приложение от простой формы заявки и показывает ключевую бизнес-функцию.

3.27 Практическая проверка демонстрационного прототипа

После реализации основных модулей была выполнена практическая проверка демонстрационного прототипа. Проверка проводилась по маршрутам приложения и по ключевым действиям пользователя. Особое внимание уделялось тому, чтобы пользовательский путь не обрывался: с главной страницы можно перейти к услугам, из услуги - к записи, из формы - получить понятный результат отправки.

При проверке каталога оценивалось, что каждая услуга содержит визуальный блок, краткое описание, длительность, стоимость, ограничения и действия. Проверялась корректность перехода «Подробнее» и «Записаться». Это важно, потому что каталог является центральным разделом публичной части и связывает информационный сценарий с формой заявки.

При проверке формы записи последовательно тестировались пустая форма, частично заполненная форма, форма без согласия, форма с одним участником и форма с несколькими участниками. Отдельно проверялось, что недоступные слоты не выглядят как обычные активные кнопки и сопровождаются причиной. Такой тест подтверждает, что бизнес-ограничения видны пользователю.

При проверке административного режима оценивалось, что вкладки не дублируют публичные страницы, а дополняют их функциями управления. Изменение фото и текста должно быть понятно администратору без знания путей проекта. Поэтому кнопки добавления файла и поля редактирования расположены рядом с соответствующими данными.

Скриншоты, включенные в пояснительную записку, были получены с актуальной версии локального приложения. Это важно для достоверности ВКР: иллюстрации отражают не абстрактный дизайн, а фактически реализованные страницы. Скриншоты показывают главную страницу, каталог, карточку услуги, форму записи, правила, галерею, контакты и административные разделы.

3.28 Практическое значение реализованной клиентской части

Практическое значение разработанной клиентской части состоит в том, что она уже может использоваться как демонстрационный стенд для обсуждения с заказчиком и командой разработки. Заказчик видит будущий пользовательский сценарий, backend-разработчик видит контракт данных, а руководитель ВКР получает подтверждение, что результат является не описанием, а работающим web-приложением.

Для предприятия проект полезен тем, что показывает, как можно объединить разрозненные сведения об услугах, фотографиях, правилах и заявках. Даже без backend клиентская часть демонстрирует будущую организацию работы: клиент отправляет структурированную заявку, администратор просматривает календарь, управляет лошадьми и изменяет правила записи.

Для дальнейшего развития важна возможность постепенного внедрения. Сначала можно подключить реальные услуги и контакты, затем реализовать прием заявок на сервере, после этого добавить загрузку фотографий и защищенную админку, а уже затем расширять календарь уведомлениями, аналитикой и личным кабинетом. Такая последовательность снижает риски и соответствует ресурсам малого предприятия.

Таким образом, практическая часть подтверждает достижение цели ВКР: разработана клиентская часть, которая учитывает специфику конюшни, поддерживает календарную предварительную запись, позволяет редактировать контент и готова к backend-интеграции. Реализация не выходит за установленные ограничения, так как не создает сервер, базу данных и промышленную авторизацию.

3.29 Резерв практического сопровождения и передачи проекта

Для передачи проекта следующему участнику команды необходимо зафиксировать, какие части frontend уже готовы, а какие должны быть заменены серверной реализацией. Готовыми считаются структура маршрутов, публичные страницы, форма записи, административный режим, TypeScript-модели, mock-данные, mock API layer и алгоритм доступности. Заменяемыми считаются хранение данных, авторизация, загрузка файлов, окончательная проверка заявки и сохранение расписания.

Backend-разработчику следует использовать текущие mock-функции как карту интеграции. Например, если компонент вызывает `getServices`, сервер должен предоставить endpoint с аналогичным набором полей. Если форма вызывает `checkAvailability`, сервер должен вернуть доступность, причины недоступности и, при необходимости, предварительные назначения ресурсов. Такой подход уменьшает риск несогласованности между частями проекта.

При передаче проекта также важно сохранить демонстрационные данные. Они нужны не только для разработки, но и для тестирования после подключения backend. На их основе можно проверить, что каталог отображается, галерея группируется,

лошади участвуют в подборе, правила закрывают слоты, а заявки появляются в календаре администратора.

Таким образом, практический результат ВКР включает не только исходный код приложения, но и подготовленную основу для командной работы. Это повышает ценность проекта: frontend можно демонстрировать отдельно, но его структура уже ориентирована на дальнейшее внедрение полноценной информационной системы предприятия.

4 ОЦЕНКА РЕЗУЛЬТАТА И ТРЕБОВАНИЯ К ИНТЕГРАЦИИ

4.1 Ожидаемый эффект внедрения

Разработанная клиентская часть позволяет предприятию представить услуги в едином цифровом канале и сократить количество повторяющихся уточнений. Клиент заранее получает сведения о стоимости, длительности, требованиях к участникам, правилах безопасности и контактах. Администратор в перспективе получает структурированные заявки вместо разрозненных сообщений.

Для малого предприятия особенно важна возможность быстро менять содержимое сайта без привлечения разработчика для каждой замены фотографии или цены. Поэтому в проект включен демонстрационный режим редактирования контента. В дальнейшем этот режим должен быть связан с серверной частью, где будут храниться данные и права доступа администратора.

4.2 Защита персональных данных и ограничения frontend-проверок

Форма записи собирает имя клиента, телефон и сведения об участниках, поэтому перед отправкой требуется согласие на обработку персональных данных. В текущей frontend-реализации данные не сохраняются в localStorage как персональные сведения клиента, а mock-отправка только имитирует создание заявки. В рабочей версии передача данных должна выполняться по HTTPS, а сервер обязан повторно валидировать все поля [4], [14].

Клиентская валидация повышает удобство интерфейса, но не рассматривается как единственная защита. Backend-разработчику необходимо реализовать проверку прав администратора, защиту endpoint, серверную проверку доступности слотов, контроль конфликтов, журналирование изменений и хранение заявок в базе данных.

4.3 Согласование с backend-разработчиком

Для интеграции требуется согласовать форматы JSON-объектов Service, BookingRequest, BookingParticipant, Horse, Booking, BookingRule, TimeSlot, AvailabilityCheckResult, Review, GalleryItem и ContactInfo. Также необходимо определить единый формат дат и времени, список допустимых статусов заявок и лошадей, правила обработки изображений и ограничения на размер загружаемых файлов.

Таблица 11 - Вопросы для согласования с backend-разработчиком

Область	Что требуется согласовать
Авторизация администратора	Способ входа, роли, хранение токена, срок действия сессии
Изображения	Endpoint загрузки файлов, допустимые форматы, максимальный размер, хранение URL
Бронирование	Серверный алгоритм доступности, атомарность подтверждения заявки, обработка конфликтов
Расписание	Формат исключений, закрытых дат, рабочих часов и перерывов
Заявки	Статусы, комментарии администратора, уведомления клиента
Персональные данные	Согласие, политика обработки, срок хранения, доступ сотрудников
Ошибки API	Единый формат сообщения для клиента без технических деталей

ЗАКЛЮЧЕНИЕ

В ходе выпускной квалификационной работы разработана клиентская часть web-приложения для малого предприятия сферы предоставления конно-спортивных услуг. Цель работы достигнута: создан адаптивный интерфейс, позволяющий клиенту изучить услуги, открыть подробную карточку, ознакомиться с правилами

безопасности, выбрать дату и время через календарный сценарий, добавить одного или нескольких участников и отправить предварительную заявку на подтверждение администратором.

В работе выполнен анализ предметной области, определены пользовательские требования, выбран технологический стек React, TypeScript, Vite и React Router, спроектирована структура страниц и компонентов. Mock-данные, TypeScript-типы и API layer вынесены отдельно от компонентов, что упрощает будущую интеграцию с сервером.

Особое внимание уделено специфике конюшни: реализована frontend-логика проверки доступности временных слотов с учетом ограниченного количества лошадей, допустимого веса наездника, статуса лошади, разрешенных услуг, существующих заявок, закрытых дат и перерыва после занятий. Пользователю отображаются понятные причины недоступности слота.

Дополнительно реализован демонстрационный административный режим. Он позволяет изменять оформление главной страницы, тексты страниц, фотографии, услуги, цены, галерею, список лошадей, календарь заявок и правила записи. Данный режим показывает будущую бизнес-функцию управления сайтом, но не заменяет полноценную серверную авторизацию.

Дальнейшее развитие проекта связано с реализацией backend API, базы данных, загрузки файлов на сервер, защищенной авторизации администратора, серверной проверки бронирований и уведомлений клиентов о статусе заявки. Разработанная клиентская часть пригодна для демонстрации в дипломном проекте и может служить основой для промышленной версии приложения.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

- 1 ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления. - Москва : Стандартинформ, 2018.
- 2 ГОСТ Р 7.0.100-2018. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая запись. Библиографическое описание. Общие требования и правила составления. - Москва : Стандартинформ, 2018.
- 3 ГОСТ Р 7.0.5-2008. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая ссылка. Общие требования и правила составления. - Москва : Стандартинформ, 2008.
- 4 Российская Федерация. Законы. О персональных данных : Федеральный закон от 27.07.2006 № 152-ФЗ : редакция, действующая на дату обращения 25.05.2026.
- 5 ISO/IEC/IEEE 29148:2018. Systems and software engineering - Life cycle processes - Requirements engineering. - Geneva : ISO, 2018.
- 6 ISO 9241-210:2019. Ergonomics of human-system interaction - Part 210: Human-centred design for interactive systems. - Geneva : ISO, 2019.
- 7 React Documentation : официальный сайт. - URL: <https://react.dev/> (дата обращения: 25.05.2026).
- 8 TypeScript Documentation : официальный сайт. - URL: <https://www.typescriptlang.org/docs/> (дата обращения: 25.05.2026).
- 9 Vite Documentation : официальный сайт. - URL: <https://vite.dev/guide/> (дата обращения: 25.05.2026).
- 10 React Router Documentation : официальный сайт. - URL: <https://reactrouter.com/> (дата обращения: 25.05.2026).
- 11 MDN Web Docs. Fetch API. - URL: https://developer.mozilla.org/docs/Web/API/Fetch_API (дата обращения: 25.05.2026).
- 12 MDN Web Docs. HTML forms. - URL: <https://developer.mozilla.org/docs/Learn/Forms> (дата обращения: 25.05.2026).
- 13 MDN Web Docs. CSS media queries. - URL: https://developer.mozilla.org/docs/Web/CSS/CSS_media_queries (дата обращения: 25.05.2026).

- 14 OWASP Application Security Verification Standard. - URL: <https://owasp.org/www-project-application-security-verification-standard/> (дата обращения: 25.05.2026).
- 15 Web Content Accessibility Guidelines (WCAG) 2.2. W3C Recommendation. - URL: <https://www.w3.org/TR/WCAG22/> (дата обращения: 25.05.2026).
- 16 Sommerville I. Software Engineering. 10th ed. - Boston : Pearson, 2016. - 816 p.
- 17 Pressman R. S., Maxim B. R. Software Engineering: A Practitioner's Approach. 9th ed. - New York : McGraw-Hill Education, 2019. - 896 p.
- 18 Фаулер М. Архитектура корпоративных программных приложений / пер. с англ. - Москва : Вильямс, 2016. - 544 с.
- 19 ГОСТ 34.602-2020. Информационная технология. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы. - Москва : Стандартинформ, 2020.
- 20 ГОСТ 19.701-90. Единая система программной документации. Схемы алгоритмов, программ, данных и систем. Условные обозначения и правила выполнения. - Москва : Издательство стандартов, 1991.

ПРИЛОЖЕНИЕ А

API-КОНТРАКТ ДЛЯ BACKEND-РАЗРАБОТЧИКА

В приложении приведен перечень endpoint, которые необходимо реализовать на серверной стороне для замены mock API layer реальными запросами.

Таблица А.1 - Основные endpoints backend API

Метод и URL	Назначение
GET /api/services	Получить каталог услуг
GET /api/services/{id}	Получить подробную карточку услуги
POST /api/bookings	Создать предварительную заявку
GET /api/bookings	Получить список заявок администратора
PATCH /api/bookings/{id}/status	Изменить статус заявки
PATCH /api/bookings/{id}/assign-horses	Назначить лошадей участникам
GET /api/horses	Получить список лошадей
POST /api/horses	Добавить лошадь
PATCH /api/horses/{id}	Изменить данные лошади
DELETE /api/horses/{id}	Удалить или деактивировать лошадь
GET /api/availability?serviceId=...&date=...	Получить доступные слоты на дату
POST /api/availability/check	Проверить доступность заявки
GET /api/booking-rules	Получить правила записи
POST /api/booking-rules	Создать правило записи
PATCH /api/booking-rules/{id}	Изменить правило записи
DELETE /api/booking-rules/{id}	Удалить правило записи
GET /api/gallery	Получить элементы галереи
POST /api/uploads	Загрузить изображение и получить URL
GET /api/contacts	Получить контактные данные

ПРИЛОЖЕНИЕ Б

ФРАГМЕНТЫ МОДЕЛЕЙ ДАННЫХ TYPESCRIPT

Ниже приведены ключевые модели, согласование которых требуется при backend-интеграции. Полные определения находятся в файле `src/types/index.ts`.

```
interface BookingParticipant { id: string; fullName: string; age: number; weightKg: number; experience: RiderExperience; comment?: string; }
```

```
interface BookingRequest { clientName: string; clientPhone: string; serviceId: string; date: string; timeSlotId: string; participants: BookingParticipant[]; comment?: string; personalDataAgreement: boolean; }
```

```
interface Horse { id: string; name: string; maxRiderWeightKg: number; allowedServiceIds: string[]; status: HorseStatus; isActive: boolean; notes?: string; }
```

```
interface BookingRule { id: string; name: string; type: BookingRuleType; isActive: boolean; config: Record<string, unknown>; }
```

ПРИЛОЖЕНИЕ В

ДОПОЛНИТЕЛЬНЫЕ СКРИНШОТЫ WEB-ПРИЛОЖЕНИЯ



ИП Орлова Н.И.
конно-спортивные услуги

[Главная](#)[Услуги](#)[Правила и безопасность](#)[Галерея](#)[Отзывы](#)[Контакты](#)[Админ](#)[Записаться](#)

г. Гурьевск - ИП Орлова Н.И.

Конно-спортивные услуги для обучения, отдыха и семейного досуга

Помогаем спокойно познакомиться с лошадьми, выбрать подходящий формат занятия и оставить предварительную заявку без лишней сложности.

[Посмотреть услуги →](#)[Записаться](#)

ИП Орлова Н.И.

УСЛУГИ

Популярные направления

Каталог построен на mock-данных и готов к подключению backend API.

[✎ Редактировать сайт](#)[⚙ Панель](#)[↗ Выйти](#)

Обучение верховой езде

Обучение верховой езде

Индивидуальные и вводные занятия с инструктором для начинающих и продолжающих.

⌚ Длительность

60 минут

💰 Стоимость

2 000 руб.

👤 Возраст

с 7 лет

[Подробнее](#)[Записаться](#)

Конная прогулка

Конная прогулка

Спокойная прогулка по территории или маршруту в сопровождении инструктора.

⌚ Длительность

45-90 минут

💰 Стоимость

от 1 800 руб.

👤 Возраст

с 10 лет

[Подробнее](#)[Записаться](#)

Индивидуальное занятие

Индивидуальное занятие

Персональная тренировка с учетом цели, возраста и уровня подготовки клиента.

⌚ Длительность

60 минут

💰 Стоимость

2 500 руб.

👤 Возраст

с 7 лет

[Подробнее](#)[Записаться](#)

ПОЧЕМУ УДОБНО

Понятный маршрут клиента



Спокойный выбор

На карточках услуг сразу видны цена, длительность и ограничения.



Безопасность на виду

Правила и требования вынесены в отдельный раздел и повторяются перед записью.



Предварительная заявка

Форма не обещает автоматическое бронирование, а честно сообщает о подтверждении администратором.

Важно перед посещением

Лошади требуют внимательного и спокойного поведения. Перед занятием проводится инструктаж, а формат подбирается под возраст и подготовку клиента.

[Правила и безопасность](#)

ОТЗЫВЫ

Клиенты отмечают заботу и спокойный темп

★★★★★

★★★★★

Рисунок В.1 - Публичная страница в административном режиме

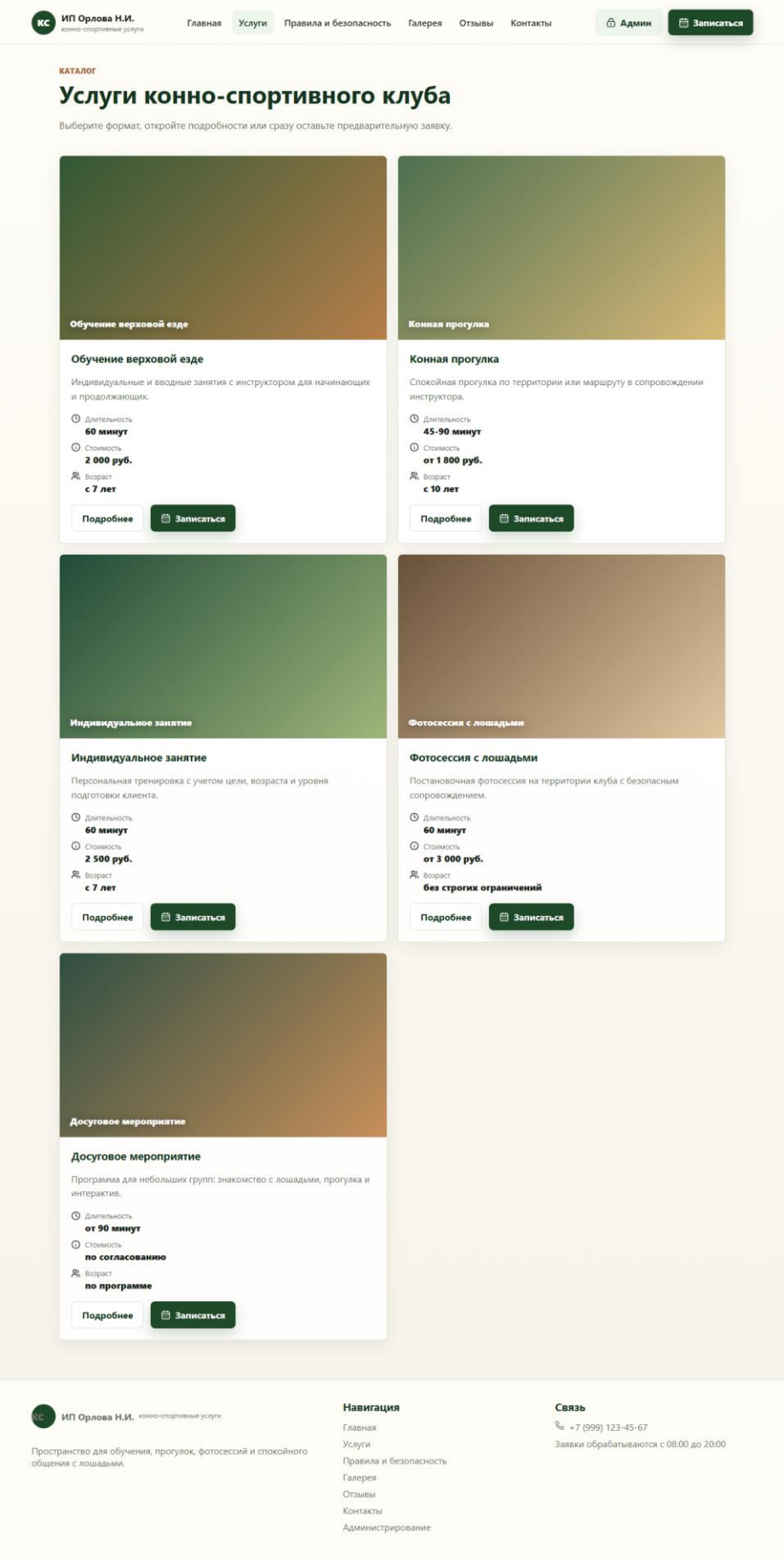


Рисунок В.2 - Полная страница каталога услуг



ГАЛЕРЕЯ

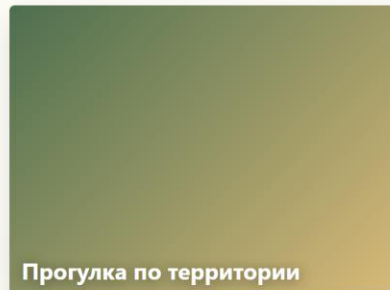
Фотографии занятий, прогулок и территории

Разделы оформлены как альбомы. В режиме администратора можно добавлять фотографии прямо в нужный блок.

1 ФОТО

Прогулки

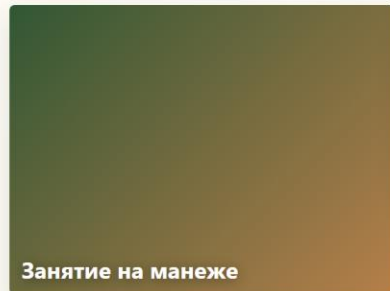
Маршруты, спокойный темп и прогулки по территории.



2 ФОТО

Занятия

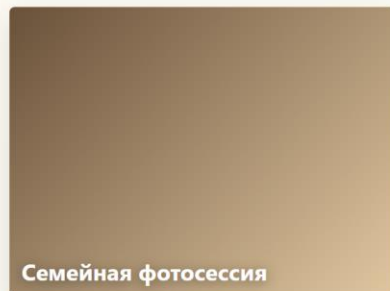
Тренировки, инструктаж и первые шаги в верховой езде.



1 ФОТО

Фотосессии

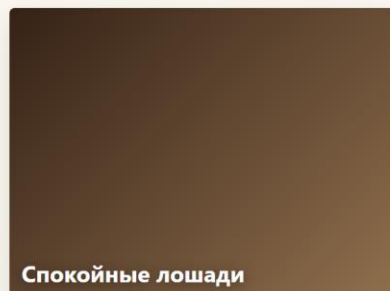
Постановочные кадры с лошадьми и семейные съемки.



1 ФОТО

Лошади

Лошади клуба, их характер и спокойная атмосфера.



1 ФОТО

Территория

Место проведения занятий, манеж и зоны отдыха.

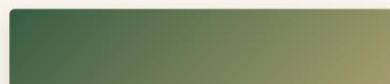


Рисунок В.3 - Полная страница галереи